

Session Spaces

Category Theory, Topoi, Sheaves, and Cohomology through SessionStream

A custom study companion prepared for a software engineer

15 August 2026

Contents

Preface	i
0.1 The mental picture	i
0.2 You are here	ii
0.3 How each chapter works	ii
0.4 Conventions	iii
0.5 A warning about software metaphors	iii
0.6 Source map	iv
1 Mathematical Universes and Engineering Boundaries	1
1.1 Why foundations matter to a software engineer	1
1.2 Objects are relative to a discourse	2
1.3 Internal and external descriptions	2
1.4 Equality is a design decision	3
1.5 A first running model: histories as words	4
1.6 What this perspective buys you	4
1.7 Exercises	5
2 Categories of Software Processes	6
2.1 The definition	6
2.2 The category Set	7
2.3 Posets as categories	7
2.3.1 Event cuts	7
2.3.2 Event histories	8
2.3.3 Invariant strength	8
2.4 Monoids as one-object categories	8
2.5 A category of typed transformations	8
2.6 A transition category	9
2.7 SessionStream is a diagram, not one object	9
2.8 Commutative diagrams as executable laws	10
2.9 Exercises	10
3 Cancellation, Isomorphism, and Distinguished Objects	11
3.1 Monic arrows	11
3.2 Epic arrows	11
3.3 Isomorphisms	12
3.4 Isomorphic objects and representation independence	12

3.5	Initial objects	13
3.6	Terminal objects	13
3.7	Duality	13
3.8	Session identity is not ordinal identity	14
3.9	Exercises	14
4	Universal Constructions in SessionStream	16
4.1	The universal-property pattern	16
4.2	Products	16
4.2.1	Product projections in SessionStream	17
4.2.2	Session decomposition	18
4.3	Coproducts	18
4.4	Equalizers	19
4.4.1	Replay agreement as an equalizer	19
4.5	Coequalizers	20
4.6	Diagrams and cones	20
4.7	Limits	21
4.7.1	Limits as compatible databases of facts	21
4.8	Colimits	22
4.9	Pullbacks	22
4.9.1	Schema validation as a pullback	22
4.9.2	Reindexing by pullback	23
4.10	Pushouts	24
4.11	Finite completeness	24
4.12	Exponentials	24
4.12.1	Handler and projection spaces	25
4.13	A complete worked limit: coherent hydration	25
4.14	When limits fail to exist in the model	26
4.15	Exercises	26
5	Subobjects, Classifiers, and the Topos Idea	28
5.1	Subobjects are embeddings up to representation	28
5.1.1	Software invariants as subobjects	29
5.2	Generalized elements	29
5.3	Characteristic functions in Set	30
5.4	Subobject classifiers	30
5.4.1	A classifier for typed admission	30
5.5	The definition of an elementary topos	31
5.6	Power objects	31
5.7	Topos thinking for API contracts	32
5.8	First encounter with sheaves	32
5.9	Exercises	32
6	Images, Reachability, and Extensionality	34
6.1	Monics as equalizers	34
6.2	Images	34
6.2.1	Reachable materialized states	35
6.3	Image as the smallest containing subobject	35

6.4	Extensionality	35
6.5	Bivalence and its limits	36
6.6	Observable equivalence	36
6.7	Exercises	36
7	Logic as Algebra of Invariants	38
7.1	Propositions as subobjects	38
7.2	Conjunction and intersection	38
7.3	Disjunction and union	39
7.4	Negation	39
7.5	Implication	39
7.6	The subobject lattice	40
7.7	Pulling invariants backward	40
7.8	Classical logic as a special case	41
7.9	Exercises	41
8	Intuitionistic Logic as Growing Information	42
8.1	Truth at a stage	42
8.2	Kripke semantics	42
8.3	Why excluded middle can fail	43
8.4	Heyting algebras of information	43
8.5	Safety and liveness	44
8.6	Local proof objects	44
8.7	Refinement of knowledge versus mutation of truth	44
8.8	Exercises	45
9	Functors, Natural Transformations, and Multiple Interpreters	46
9.1	Functors preserve categorical structure	46
9.2	Histories acting on states	46
9.3	Covariant and contravariant behavior	47
9.4	Natural transformations	47
9.4.1	Projection-version migration	48
9.4.2	Encoding transformations	49
9.5	Natural isomorphism and equivalence	49
9.6	Functor categories	49
9.7	A caution about projections	49
9.8	Exercises	50
10	Presheaf Topoi and Stage-Indexed Validity	51
10.1	Variable sets	51
10.2	Stage-indexed truth	51
10.3	Subpresheaves	52
10.4	The Yoneda viewpoint	52
10.5	Parameter sufficiency as factorization	52
10.6	Context design is part of the theorem	53
10.7	Exercises	54
11	Internal Languages and Schemas	55

11.1	Signatures before implementations	55
11.2	A small SessionStream language	55
11.3	Terms as arrows	56
11.4	Formulas as subobjects	57
11.5	Partial elements	57
11.6	Internal versus external claims	57
11.7	Completeness is relative to a logic and model class	58
11.8	Exercises	58
12	Choice, Natural Numbers, and Recursive Folds	59
12.1	The categorical axiom of choice	59
12.2	Natural numbers objects	59
12.3	Ordinals are not automatically an NNO	60
12.4	Primitive recursion and folds	60
12.5	Induction as proof over construction	60
12.6	Recursion and snapshots	61
12.7	Exercises	61
13	Presheaves as Contextual APIs	62
13.1	The base category of contexts	62
13.2	Definition of a presheaf	63
13.3	A cut presheaf of histories	63
13.4	A snapshot presheaf	64
13.5	A presheaf of observations	64
13.6	Sections and fibers	65
13.7	Presheaves of constraints	65
13.8	When restriction is not mere field deletion	65
13.9	Presheaf morphisms	66
13.10	Exercises	66
14	Sheaves, Covers, and Snapshot-Plus-Live Gluing	67
14.1	Covers	67
14.2	Matching families	67
14.3	The sheaf condition	68
14.4	Snapshot plus live suffix as a cover	68
14.5	Existence, uniqueness, and explicit failure	69
14.5.1	Failure of existence	70
14.5.2	Failure of uniqueness	70
14.6	The separatedness condition	70
14.7	Sites and Grothendieck topologies	70
14.8	Example sites for SessionStream	71
14.8.1	Cut site	71
14.8.2	Observer site	71
14.8.3	Transaction site	71
14.9	Sheafification	71
14.10	Local truth and Kripke-Joyal semantics	72
14.10.1	Software reading	72
14.11	A detailed hydration matching condition	72

14.12	Why pairwise agreement may not be enough	73
14.13	Exercises	73
15	Adjunctions, Reindexing, and Quantifiers	75
15.1	The hom-set correspondence	75
15.2	Free and forgetful constructions	76
15.3	Product and exponential adjunction	76
15.4	Reindexing predicates	76
15.5	Existential quantification as a left adjoint	77
15.5.1	Software example	77
15.6	Universal quantification as a right adjoint	77
15.7	Parameter sufficiency revisited	78
15.8	Pullback functors between slice categories	79
15.9	Unit and counit as round-trip maps	79
15.10	Exercises	79
16	Geometric Morphisms and Architectural Translation	81
16.1	Preservation and reflection	81
16.2	Geometric morphisms	81
16.3	Software analogy: changing worlds of observation	82
16.4	Geometric logic	82
16.5	Theories as sites	83
16.6	Refinement as a functor with laws	83
16.7	Exercises	83
17	From Covers to Multidimensional Shapes	85
17.1	The nerve of a cover	85
17.2	Why triangles matter	85
17.3	Cycles and holes	86
17.4	Architecture as a simplicial complex	86
17.5	Example: snapshot consistency complex	87
17.6	The nerve is not the data	88
17.7	Refining the cover	88
17.8	Exercises	88
18	Cochains, Coboundaries, and Cohomology	89
18.1	Why algebra enters	89
18.2	Čech cochains	89
18.3	The first coboundary	90
18.4	The second coboundary	90
18.5	First cohomology	90
18.6	A cycle graph	91
18.7	A filled triangle	92
18.8	Higher cohomology	92
18.9	Sheaf cohomology versus ordinary topology	92
18.10	Obstruction logic and its limits	92
18.11	Exercises	93

19 Cohomological Case Studies for SessionStream	94
19.1 Case study: ordinal convention holonomy	94
19.2 Case study: atomic projection progress	95
19.3 Case study: consistent-cut snapshots	95
19.4 Case study: stable retry identity	96
19.5 Case study: schema registry as a pullback, not a hole	96
19.6 Case study: observer traces	97
19.7 A worked four-cycle	97
19.8 What a useful tool should say	98
19.9 Exercises	98
20 Building a Local-to-Global Checker	99
20.1 Two layers, not one	99
20.1.1 Layer A: set-valued consistency	99
20.1.2 Layer B: additive diagnostics	99
20.2 A declarative model	99
20.3 Verifying presheaf laws	100
20.4 Checking matching families	100
20.5 Searching for global sections	101
20.6 Building coboundary matrices	101
20.7 A Go-oriented sketch	102
20.8 Cellular sheaves for heterogeneous data	102
20.9 Integration with traces	103
20.10A staged research program	103
20.11Capstone project	103
20.12Exercises	104
A Proof and Modeling Patterns	105
A.1 Proving a category	105
A.2 Proving a universal property	105
A.3 Uniqueness up to unique isomorphism	105
A.4 Proving functoriality	106
A.5 Proving naturality	106
A.6 Proving presheaf laws	106
A.7 Proving the sheaf condition	107
A.8 Computing low-dimensional cohomology	107
A.9 Model audit questions	107
B Selected Exercise Hints and Solutions	109
B.1 Solution to 1.2	109
B.2 Solution to 2.1	109
B.3 Solution to 3.6	109
B.4 Solution to 4.2	109
B.5 Solution to 4.3	110
B.6 Solution to 4.6	110
B.7 Solution to 5.4	110
B.8 Hint for 6.2	110
B.9 Solution to 7.4	111

B.10 Solution to 8.2	111
B.11 Solution to 9.1	111
B.12 Solution to 10.3	111
B.13 Solution to 12.1	112
B.14 Solution to 14.1	112
B.15 Solution to 15.2	112
B.16 Solution to 18.1	113
B.17 Solution to 18.2	113
B.18 Solution to 18.3	113
B.19 Solution to 18.4	114
B.20 Solution to 19.1	114
C Study Plan and Working Notebook	115
C.1 A twenty-session route	115
C.2 Notebook page template	116
C.3 Diagram drills	116
C.4 Proof-writing standard	117
D Glossary	118
E Source Notes and Further Reading	121
E.1 Primary pedagogical source	121
E.2 SessionStream source snapshot	121
E.3 Additional mathematical references	122
E.4 Copyright and scope note	122
Closing Perspective	123

Preface

This is an original, software-grounded companion to Robert Goldblatt’s *Topoi: The Categorical Analysis of Logic*. It follows the dependency order and much of the conceptual itinerary of that book, but it does not reproduce its exposition or exercises. Definitions are restated in new language, examples are rebuilt around event-driven software, and the exercises are newly written.

The central running system is [sessionstream](#), read at repository commit [d62dca9f5efa](#) from 13 August 2026 (the full commit is recorded in Appendix E). The principal source paths are the [Hub](#), event and projection types, hydration interfaces and SQLite store, ordinal assignment, WebSocket snapshot-before-live protocol, and the chat demonstration. The Architecture Garden analysis at [parc.yolo.scapegoat.dev](#) supplies additional explicit laws: per-session serializability, consistent-cut snapshots, stable retry identity, atomic projection progress, deterministic replay, and snapshot-plus-suffix completeness.

Goldblatt’s book is about categorical logic rather than category theory in isolation. This companion keeps that emphasis. Category theory is introduced as a method for replacing hidden element-level implementation details with observable relationships, universal properties, and transformations that remain meaningful across representations. Topos theory then turns those constructions into a setting where predicates, truth values, quantifiers, and local reasoning can be treated structurally.

Cohomology is not developed in Goldblatt’s book. Chapters 17-20 therefore form a clearly marked supplement. They build on the preceding treatment of limits, presheaves, sheaves, and local-to-global reasoning, then introduce nerves, cochains, coboundaries, and low-dimensional cohomology as tools for studying the shape of compatibility data.

0.1 The mental picture

A SessionStream execution is never seen from one omniscient place. A handler sees a command and a publisher. A projector sees an event and a timeline view. A store sees rows and cursors. A WebSocket subscriber sees a snapshot followed by live batches. An operator sees traces. A browser sees decoded transport frames. These views overlap, but no view is identical to the complete execution.

The main mathematical question of this book is therefore:

When do locally meaningful observations assemble into one coherent global execution?

Category theory gives the language of the observations and their relationships. Limits describe compatible assemblies. Presheaves describe how information restricts from larger contexts to smaller ones. Sheaves say when compatible local information glues uniquely. Cohomology can sometimes detect persistent obstruction patterns around loops of overlap.

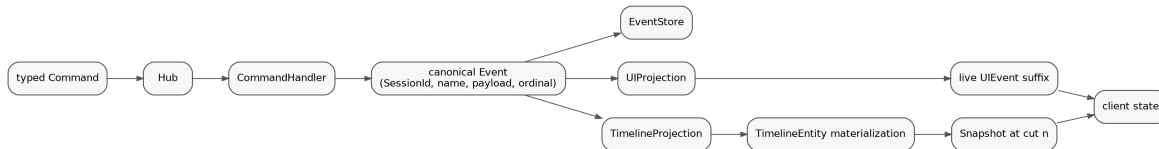


Figure 1: The running architecture. A command produces a canonical event; independent projections derive live and durable views; reconnects combine a snapshot with a future suffix.

0.2 You are here

You reported being at the limits section of Goldblatt Chapter 3, which is §3.11 in the supplied edition. That is the correct point to slow down. Products, equalizers, terminal objects, and limits are not a miscellaneous collection of constructions. They are the first appearance of the general pattern that later becomes:

- pullbacks and subobjects;
- finite completeness in a topos;
- the sheaf gluing condition as a limit;
- reindexing and quantifiers through adjunctions;
- compatible observations and obstruction theory.

A practical route is:

1. Read Chapters 1-3 here quickly, doing only the starred exercises.
2. Work through Chapter 4 slowly. Re-draw every universal-property diagram from memory.
3. Continue through Chapters 5-12 with one SessionStream notebook example per section.
4. Spend substantial time on Chapters 13-15. These contain the presheaf/sheaf bridge you were originally looking for.
5. Treat Chapters 17-20 as a second pass, not as prerequisites for understanding sheaves.

0.3 How each chapter works

Each chapter contains five recurring components.

Source alignment identifies the corresponding part of Goldblatt. This tells you what is source-derived in organization and conceptual emphasis.

Core mathematics gives definitions and small proofs. Definitions are intended to be usable, not merely suggestive.

SessionStream translation constructs a model from actual concepts in the repository. The translation is sometimes exact and sometimes analogical; the text labels that distinction.

Shape intuition gives the geometric or multidimensional picture. These pictures are aids, not replacements for definitions.

Exercises move through four levels:

- *recognition*: identify a categorical structure;
- *calculation*: construct an arrow, limit, or restriction;
- *proof*: verify a universal property or law;
- *design*: apply the concept to SessionStream or another system.

Selected hints and solutions appear in Appendix B. A problem marked **[proof]** is worth writing in complete mathematical prose. A problem marked **[lab]** is intended for code or trace data. A problem marked **[research]** may not have one canonical answer.

0.4 Conventions

Composition is written in the mathematical order:

$$g \circ f : A \rightarrow C$$

means first run $f : A \rightarrow B$, then $g : B \rightarrow C$. In code this often appears in the opposite visual order:

`c := g(f(a))`

Set denotes the category of sets and total functions. **1** denotes a terminal object; in **Set**, any singleton is terminal. **0** denotes an initial object; in **Set**, the empty set is initial.

For a category $\mathcal{C}$, the set of arrows from A to B is written

$$\mathcal{C}(A, B) \quad \text{or} \quad \text{Hom}_{\mathcal{C}}(A, B).$$

For a fixed session s , E_s^* denotes the set of finite event histories for that session. Concatenation is written by juxtaposition, and ϵ is the empty history.

A prefix relation is written

$$x \preceq y \quad \Longleftrightarrow \quad \exists z \, xz = y.$$

The phrase *global section* is used in its sheaf-theoretic sense: a section over the largest context under discussion. It does not mean a global variable.

0.5 A warning about software metaphors

There is no single canonical “category of software”. A category is a model chosen for a purpose. You must specify:

1. what the objects are;
2. what the arrows are;
3. when arrows compose;
4. what equality of arrows means;
5. why identity and associativity hold.

Side effects, exceptions, concurrency, nondeterminism, resource limits, and observational equivalence can all force a different choice of category. When this book says “model a projection as an arrow,” it means that we have abstracted enough operational detail to obtain a deterministic mathematical map. It does not claim that arbitrary Go functions form a simple category under all runtime observations.

0.6 Source map

The Goldblatt alignment is:

This companion	Goldblatt
Chapters 1-2	Chapters 1-2
Chapters 3-4	Chapter 3
Chapters 5-6	Chapters 4-5
Chapters 7-8	Chapters 6-8
Chapters 9-12	Chapters 9-13
Chapters 13-14	Chapter 14
Chapter 15	Chapter 15
Chapter 16	Chapter 16
Chapters 17-20	Supplemental material beyond Goldblatt

The most important SessionStream files for this edition are:

```
pkg/sessionstream/types.go
pkg/sessionstream/hub.go
pkg/sessionstream/projection.go
pkg/sessionstream/hydration.go
pkg/sessionstream/ordinals.go
pkg/sessionstream/consumer.go
pkg/sessionstream/schema.go
pkg/sessionstream/hydration/sqlite/store.go
pkg/sessionstream/transport/ws/server.go
examples/chatdemo/chat.go
proto/sessionstream/v1/transport.proto
```

Chapter 1

Mathematical Universes and Engineering Boundaries

Source alignment: Goldblatt, Chapter 1, especially the contrast between set-membership descriptions and structural descriptions.

1.1 Why foundations matter to a software engineer

A software system is usually introduced by listing its data structures. `SessionStream` appears to begin with `Command`, `Event`, `UIEvent`, `TimelineEntity`, and `Snapshot`. That is analogous to beginning mathematics with sets and their members.

But a list of structures does not yet explain the architecture. The architecture lies in relationships:

- a command is routed to a handler;
- a handler publishes events;
- an event is interpreted by projections;
- a timeline entity is applied to a store;
- a snapshot represents a cut of materialized history;
- a live batch is delivered only to matching subscriptions.

Goldblatt's opening move is to question whether membership must be the primitive language of mathematics. Category theory proposes that arrows and composition can be primary instead. The engineering analogue is to resist explaining a component solely by its fields and internal algorithm. Explain it by the arrows it admits, the diagrams it must make commute, and the universal properties it satisfies.

This is not anti-data. Objects still exist. It is a change in what counts as a stable explanation.

Consider these two descriptions of a snapshot.

Representation-level description

```
type Snapshot struct {
```

```

    SessionId      SessionId
    SnapshotOrdinal uint64
    Entities       []TimelineEntity
}

```

Structural description

A snapshot at cut n is an observation of a session state that is compatible with the event prefix through n , and that can be combined with every accepted live suffix strictly after n to reconstruct later client state.

The first is necessary for implementation. The second survives a change from SQLite to another store, a change in serialization, and many changes in entity layout. Category theory is largely a discipline for finding the second kind of description.

1.2 Objects are relative to a discourse

Goldblatt treats a category as a universe of mathematical discourse. The objects and arrows depend on what is being discussed.

In software, the same runtime entity can appear as a different object in different categories.

- A protobuf message can be an object in a category of schemas and schema-preserving transformations.
- The same message value can be an object in a category of runtime values and pure functions.
- Its encoded bytes can be an object in a category of wire representations and codecs.
- The event carrying it can be an object in a category of session histories and prefix extensions.

No contradiction is involved. A model selects some distinctions and suppresses others.

A useful criterion is:

A categorical model is good when its arrow equality matches the observations relevant to the law being studied.

For a codec law, byte-for-byte equality may matter. For a client rendering law, two protobuf encodings may be considered equal when they decode to the same semantic value. For a concurrency proof, timing and order may matter even when final values agree.

1.3 Internal and external descriptions

An *internal* description talks about constituents:

$$A \times B = \{(a, b) : a \in A, b \in B\}.$$

An *external* description talks about arrows into and out of the object. A product is characterized by two projections and a unique mediating arrow. Chapter 4 develops that example fully.

The same distinction appears in API design.

An internal description of a valid event says:

```
name is a nonempty string
payload is a protobuf message
session id is nonempty
payload descriptor happens to match the registered descriptor
```

An external description can identify a valid event as a pullback of two maps into a common descriptor space. The validity condition is no longer an ad hoc conjunction; it is the universal object of matching names and payload types. We construct this in Chapter 4.

External descriptions are especially valuable when:

- representations change;
- several implementations must satisfy one contract;
- a property must be transported through a functor;
- the system has many partial observers;
- local facts must be assembled into a global fact.

1.4 Equality is a design decision

Mathematics often treats functions extensionally:

$$f = g \iff \forall x, f(x) = g(x).$$

Go function values do not support general equality, and two implementations with the same outputs may differ in latency, allocation, logging, I/O, or failure behavior. Therefore a category of software functions must state which effects are abstracted away.

For example, a projection can be modeled as a pure arrow

$$p : S \times E \rightarrow S \times U^*$$

only under an abstraction in which:

- the prior view S and event E contain every relevant input;
- clocks, randomness, mutable globals, and network calls are absent or represented explicitly;
- the output state and UI word are the observations that define equality.

This explains the SessionStream deterministic replay obligation. If live projection and replay use the same explicit coordinates but still differ, either the implementation violates the modeled arrow or the model omitted a coordinate.

1.5 A first running model: histories as words

For a fixed session s , let E_s be the set of admitted event values. A finite history is a word

$$h = e_1 e_2 \cdots e_n \in E_s^*.$$

The empty word is ϵ . Concatenation is associative:

$$(xy)z = x(yz).$$

This is already a one-object category: the sole object represents the state space, and every history is an endomorphism-shaped program fragment. Composition is concatenation. The identity arrow is ϵ .

A stateful fold gives an action of this history monoid on states:

$$\text{fold}(S_0, xy) = \text{fold}(\text{fold}(S_0, x), y).$$

That equation is not merely a convenient implementation property. It is the compatibility between monoid composition and interpretation.

SessionStream does not imply that events commute. Usually

$$e_1 e_2 \neq e_2 e_1$$

as words, and their folds can differ. The category remembers order through composition.

1.6 What this perspective buys you

The categorical perspective can separate four questions that are often mixed together.

1. **Well-formedness:** Is a value admitted by the schema and type contract?
2. **Composability:** Can two transformations legally be composed?
3. **Equivalence:** Are two representations or implementations the same for the observations under study?
4. **Universality:** Is an object the canonical solution to a whole family of compatible problems?

The fourth question is the distinctive one. A product is not merely a pair type. A pullback is not merely a database join. A limit is not merely an aggregate. Each is characterized by the unique factorization of every competing solution.

1.7 Exercises

1.1 [recognition]. Give three different choices of objects and arrows for modeling `SessionStream`. For each, state what equality of arrows should mean.

1.2 [proof]. Prove associativity of concatenation for finite event words. Then explain why this proof does not prove that concurrent publication is serializable.

1.3 [design]. A projector reads `time.Now()` and inserts the timestamp into a timeline entity. List two ways to obtain a mathematically deterministic arrow. Which way better supports replay?

1.4 [recognition]. Classify each statement as primarily internal or external.

1. A `TimelineEntity` has fields `Kind`, `Id`, and `Payload`.
2. Every update of an existing entity factors through its stable `(Kind, Id)` identity.
3. A snapshot contains a list of entities.
4. A snapshot glues with every later suffix satisfying the ordinal boundary.

1.5 [research]. Choose one architectural promise in your own software. Write a representation-level definition and a structural definition. Identify which changes would preserve the structural definition.

Chapter 2

Categories of Software Processes

Source alignment: Goldblatt, Chapter 2: functions, composition, categories, abstraction, and basic examples.

2.1 The definition

Definition. A category $\mathcal{C}$ consists of objects, arrows, domain and codomain assignments, a partially defined composition operation, and an identity arrow $1_A : A \rightarrow A$ for each object A , satisfying associativity and identity laws.

If

$$A \xrightarrow{f} B \xrightarrow{g} C,$$

then the composite is

$$g \circ f : A \rightarrow C.$$

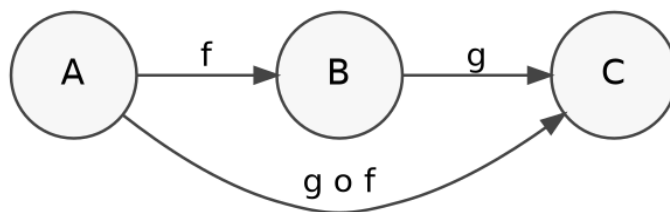


Figure 2.1: Composition.

The laws are

$$h \circ (g \circ f) = (h \circ g) \circ f$$

whenever the composites make sense, and

$$1_B \circ f = f = f \circ 1_A.$$

These laws say that a long pipeline has a stable meaning independent of parenthesization, and that doing nothing before or after a transformation changes nothing.

The laws are equations of arrows, so arrow equality must already have been chosen.

2.2 The category **Set**

Set has sets as objects and total functions as arrows. It is the primary motivating example for Goldblatt and for elementary topos theory.

Many software examples initially live in **Set** after abstraction:

- the set of valid commands;
- the set of events;
- the set of timeline states;
- a deterministic decoder as a function from valid bytes to messages;
- a projection as a function between mathematical state sets.

Partial functions require adjustment. You can represent a partial function $A \rightarrow B$ as a total function

$$A \rightarrow B + \{\text{error}\},$$

or work in a category designed for partial maps. Exceptions and cancellation should not be silently ignored.

2.3 Posets as categories

A preorder $(P, \leq)$ becomes a category by declaring that there is one arrow

$$p \rightarrow q$$

exactly when $p \leq q$. Reflexivity supplies identities; transitivity supplies composition. Because there is at most one arrow between two objects, all parallel arrows are equal.

This example is crucial because many information structures are ordered.

2.3.1 Event cuts

For one session, let the objects be natural-number cuts

$$0, 1, 2, \dots$$

and let there be an arrow $m \rightarrow n$ when $m \leq n$. The arrow means that cut n contains at least as much event-prefix information as cut m .

2.3.2 Event histories

Let objects be histories, with an arrow

$$x \rightarrow y$$

when $x \preceq y$. This category distinguishes histories with the same length but different events. An arrow records extension by some suffix.

2.3.3 Invariant strength

Let objects be predicates and put an arrow $P \rightarrow Q$ when P implies Q . Then products will become conjunctions and exponentials will relate to implication. This is the order-theoretic doorway into Heyting algebras.

2.4 Monoids as one-object categories

A monoid $(M, \cdot, e)$ is a category with one object $*$. Every element $m \in M$ is an arrow $* \rightarrow *$; composition is multiplication; e is the identity.

For `SessionStream`, E_s^* is the free monoid of histories. A projection fold is then a monoid action or, equivalently, a functor from this one-object category into a category of state transformations.

This viewpoint turns a long history from a passive list into a composed arrow.

2.5 A category of typed transformations

Let objects be selected semantic types:

```

Command
Event
TimelineState
Snapshot
ClientState
WireFrame

```

Let arrows be pure total transformations between them. Potential arrows include:

$$\text{validate} : \text{RawCommand} \rightarrow \text{Command} + \text{Error},$$

$$\text{project} : \text{TimelineState} \times \text{Event} \rightarrow \text{TimelineState},$$

$$\text{encode} : \text{Snapshot} \rightarrow \text{WireFrame}.$$

This forms a category only after error values and effects are treated consistently. A Go function that panics is not a total arrow under ordinary value semantics. A function that mutates hidden state may not compose extensionally as expected.

A safer approach is to define arrows as explicit effectful computations in a category appropriate to those effects. Later category theory describes this using constructions such as Kleisli categories, although that topic is outside Goldblatt’s route and outside the core of this book.

2.6 A transition category

An alternative model keeps operational steps visible.

- Objects are runtime states.
- An arrow $x \rightarrow y$ is an allowed finite trace taking state x to state y .
- Composition concatenates traces.
- The empty trace is an identity.

This is a category when traces compose associatively and trace equality is chosen coherently. If two different traces between the same states remain distinct, the category can contain many parallel arrows. That is useful for observer and refinement work: final-state equality does not erase evidence about how the state was reached.

A heartbeat implementation, for example, can have arrows labeled by timer firing, ping enqueue, successful write acknowledgement, pong reception, and timeout. The state-transition category retains the distinction between “ping intended” and “ping actually written.”

2.7 SessionStream is a diagram, not one object

It is tempting to say “SessionStream is a category.” That is too vague. A more useful statement is:

SessionStream supplies several categories and functors between them, arranged in an architecture diagram.

Examples include:

- a category of command/event schemas;
- a category of session histories;
- a category of timeline states and transitions;
- a category of transport frames;
- a category of observation contexts;
- a preorder of cuts or checkpoints.

The architecture itself can be studied as a diagram whose nodes are these domains and whose arrows are validation, projection, storage, encoding, and restriction operations.

2.8 Commutative diagrams as executable laws

A diagram commutes when every directed path with the same start and finish gives the same arrow.

For deterministic replay, one desired square is:

$$\begin{array}{ccc}
 (H_0, e_1 \cdots e_n) & \xrightarrow{\text{live application}} & S_n \\
 \downarrow \text{persist/reload} & & \downarrow \text{identity} \\
 (H_0, e_1 \cdots e_n) & \xrightarrow{\text{rebuild}} & S_n.
 \end{array}$$

The square says that live materialization and replay are equal under the chosen state observation.

A test can sample this law. A proof can establish it for a model. A refinement argument can connect the model to the Go implementation.

2.9 Exercises

2.1 [proof]. Verify that the prefix relation on finite histories is reflexive, transitive, and antisymmetric. Conclude that it defines a poset category.

2.2 [design]. Define a category whose objects are SessionStream schemas and whose arrows are backward-compatible migrations. State a composition law. Identify at least one reason the proposed arrows may fail to be closed under composition.

2.3 [recognition]. Model a bounded queue as a transition category. Give two distinct parallel arrows between the same start and end states that should not be identified if drop accounting matters.

2.4 [proof]. In a preorder category, show that every diagram with the same source and target commutes.

2.5 [lab]. Extract a trace from a SessionStream test or log. Represent each runtime state as a node and each observed transition as an edge. Determine whether concatenating adjacent edges always yields another valid trace in your model.

2.6 [research]. Write one commutative square for each of the following: codec round-trip, snapshot reconstruction, schema validation, and projection replay. State exactly what equality means in each square.

Chapter 3

Cancellation, Isomorphism, and Distinguished Objects

Source alignment: Goldblatt §§3.1-3.7: monic, epic, and iso arrows; isomorphic objects; initial and terminal objects; duality.

3.1 Monic arrows

Definition. An arrow $m : A \rightarrow B$ is **monic** if it is left-cancellable: for all $g, h : X \rightarrow A$,

$$m \circ g = m \circ h \implies g = h.$$

In **Set**, monic arrows are exactly injective functions. But the definition is arrow-theoretic and can behave differently in other categories.

A software interpretation in a concrete function category is *information preservation*. If `encode : Event -> Bytes` is monic, then two event-producing computations cannot become indistinguishable after encoding. This requires an injective encoding over the modeled event set.

A hash is generally not monic: collisions identify distinct inputs. A projection from a full event to a UI event is usually not monic: several backend events may intentionally have the same UI observation.

Monicity is relative to the category. If the domain category already identifies events by a coarse observational equivalence, a lossy representation may still be monic with respect to that quotient.

3.2 Epic arrows

Definition. An arrow $e : A \rightarrow B$ is **epic** if it is right-cancellable: for all $g, h : B \rightarrow X$,

$$g \circ e = h \circ e \implies g = h.$$

In **Set**, epic arrows are exactly surjective functions. Epics say that downstream behavior is determined by behavior on the image of e .

A parser from valid wire frames onto all semantic frame variants may be epic when every semantic value has some accepted encoding. Canonical encoders are often not surjective onto all byte strings, but the associated decoder may be surjective onto semantic values.

Do not equate epic with surjective outside concrete set-like categories. Goldblatt emphasizes this early because arrow definitions are meant to survive changes of universe.

3.3 Isomorphisms

Definition. An arrow $f : A \rightarrow B$ is an **isomorphism** if there is an arrow $f^{-1} : B \rightarrow A$ such that

$$f^{-1} \circ f = 1_A, \quad f \circ f^{-1} = 1_B.$$

An isomorphism is a reversible change of representation in the category. It is stronger than having a one-way migration and stronger than being both monic and epic in an arbitrary category.

A codec pair is an isomorphism only if both round trips are identities under the chosen equality:

$$\text{decode}(\text{encode}(x)) = x,$$

$$\text{encode}(\text{decode}(b)) = b.$$

The second law fails for many practical codecs because decoding then re-encoding canonicalizes the bytes. In that case semantic values may be isomorphic to *canonical encodings*, not to all accepted encodings.

This distinction is productive. It tells you which object should appear in the model.

3.4 Isomorphic objects and representation independence

Objects A and B are isomorphic when there is an isomorphism between them. Category theory normally treats isomorphic objects as structurally interchangeable, while keeping them technically distinct.

Examples:

- a timeline entity map and a sorted list with unique keys can be isomorphic if each representation determines the other;
- a snapshot protobuf and a domain snapshot can be isomorphic under a lossless codec;
- an event history and a compressed history are not isomorphic if compression discards information needed for replay;

- a live UI stream and a final snapshot are not isomorphic because the live stream contains temporal observations not generally recoverable from final state.

“Same data” should therefore be replaced by a specific isomorphism claim with explicit round-trip laws.

3.5 Initial objects

Definition. An object 0 is **initial** if for every object A there is exactly one arrow

$$0 \rightarrow A.$$

In **Set**, the empty set is initial. In a category of types and total functions, an uninhabited type plays this role: from an impossible value one can produce a value of any type, because there is no input case to handle.

In a transition category, an initial object can represent a unique origin state from which every modeled execution has one canonical initialization arrow. That is a stronger property than merely having a startup state; uniqueness matters.

3.6 Terminal objects

Definition. An object 1 is **terminal** if for every object A there is exactly one arrow

$$A \rightarrow 1.$$

In **Set**, any singleton is terminal. The unique function forgets all information.

In software, the unit type is terminal in a category of pure total functions. Every computation can discard its result:

```
func discard[A any](A) struct{} { return struct{}{} }
```

Terminal objects encode a universal form of forgetting. They also allow generalized elements: an element of A is represented by an arrow $1 \rightarrow A$. This becomes important for categorical logic, where “elements” are replaced by arrows from arbitrary contexts, not only from 1 .

3.7 Duality

Every category $\mathcal{C}$ has an opposite category $\mathcal{C}^{op}$ with the same objects and all arrows reversed. A theorem has a dual obtained by reversing arrows and composition order.

Pairs of dual notions include:

Notion	Dual
monic	epic
initial	terminal
product	coproduct
equalizer	coequalizer
pullback	pushout
limit	colimit

Duality is not a vague symmetry. It is a method for generating definitions and proofs.

For information systems, reversal often exchanges “assemble” and “decompose,” or “consume” and “produce.” The analogy can help, but the formal dual is always defined by reversing arrows in a specified category.

3.8 Session identity is not ordinal identity

A useful application of these distinctions concerns `SessionId` and `Ordinal`.

An ordinal is intended as a per-session sequence coordinate. It is not necessarily a globally monic identifier of events: two sessions can have the same ordinal, and bus redelivery may assign a new ordinal to the same logical event. The pair

(`SessionId`, `Ordinal`)

is a stronger coordinate, but it still identifies delivery position rather than semantic event identity unless the architecture guarantees otherwise.

A stable `EventId` would define a different arrow into an identity space. Asking whether that arrow is monic forces a precise question: can two distinct logical events share the same ID?

3.9 Exercises

3.1 [proof]. Show that every identity arrow is monic, epic, and iso.

3.2 [proof]. Show that the composite of two monic arrows is monic. State and prove the dual result.

3.3 [calculation]. Let `encode` map semantic snapshots to canonical JSON bytes, while `decode` accepts canonical and noncanonical whitespace variants. Determine which round-trip equations hold. Identify the objects between which an isomorphism exists.

3.4 [design]. Propose an event identity type for `SessionStream`. Separate semantic identity, delivery identity, and order coordinate. State which maps should be monic.

3.5 [recognition]. Identify initial and terminal objects in each category:

1. sets and total functions;
2. event cuts ordered by $\leq$;
3. predicates ordered by implication;
4. one-object category of event histories.

3.6 [proof]. Initial objects are unique up to a unique isomorphism. Write the standard proof using the two unique arrows.

3.7 [research]. Find an engineering claim phrased as “these representations are equivalent.” Rewrite it as an explicit isomorphism, equivalence weaker than isomorphism, or one-way refinement. Justify the choice.

Chapter 4

Universal Constructions in Session-Stream

Source alignment: Goldblatt §§3.8-3.16. This chapter is the custom counterpart to the point you have currently reached, especially §3.11 on limits and colimits.

4.1 The universal-property pattern

A universal construction is not characterized primarily by what it contains. It is characterized by how every competing construction maps to or from it.

A typical limiting universal property has this form:

1. specify a pattern of arrows, called a diagram;
2. define cones over that diagram;
3. select a cone through which every other cone factors;
4. require the factorization arrow to be unique.

The unique factorization is the source of canonicity. If two objects satisfy the same universal property, they are uniquely isomorphic in the relevant sense.

This is the right mental model for the phrase “canonical solution.” It does not mean a favorite implementation. It means an object determined up to unique isomorphism by its relation to every admissible candidate.

4.2 Products

Definition. A product of A and B is an object $A \times B$ with projections

$$\pi_A : A \times B \rightarrow A, \quad \pi_B : A \times B \rightarrow B,$$

such that for every pair $f : X \rightarrow A, g : X \rightarrow B$, there is exactly one arrow

$$\langle f, g \rangle : X \rightarrow A \times B$$

with $\pi_A \circ \langle f, g \rangle = f$ and $\pi_B \circ \langle f, g \rangle = g$.

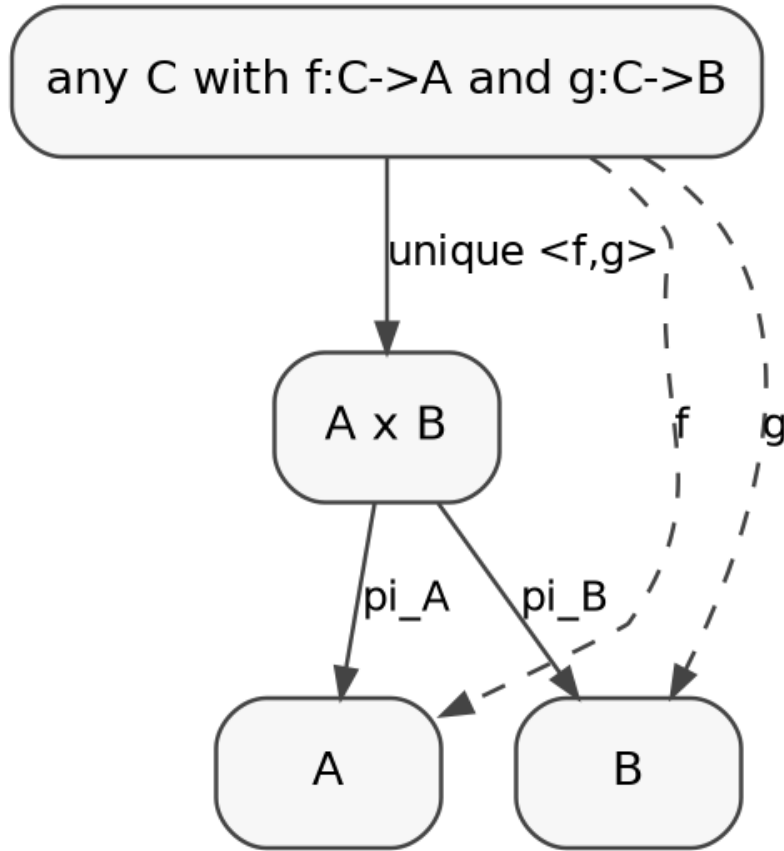


Figure 4.1: The universal property of a product.

In **Set**, this is the ordinary Cartesian product. The universal property says that giving one function into a pair is exactly the same as giving its two component functions:

$$\text{Set}(X, A \times B) \cong \text{Set}(X, A) \times \text{Set}(X, B).$$

4.2.1 Product projections in SessionStream

Suppose one canonical event is interpreted by independent UI and timeline projections:

$$p_U : S \times E \rightarrow U^*,$$

$$p_T : S \times E \rightarrow T.$$

Their product interpretation is

$$\langle p_U, p_T \rangle : S \times E \rightarrow U^* \times T.$$

The product does not imply that the two projections are operationally independent. If either performs hidden effects or reads mutable shared state, the mathematical product model is inaccurate. But under pure semantics, it says one input can be observed through both interpreters without identifying their output types.

4.2.2 Session decomposition

The intended global state often has a product shape

$$S = \prod_{s \in \text{Sessions}} S_s.$$

An event for session a should change the a -component and leave every $b \neq a$ component unchanged. This is a noninterference law. For an infinite family, this is a possibly infinite product, not merely a binary product.

4.3 Coproducts

The dual construction is the coproduct.

Definition. A coproduct of A and B is an object $A + B$ with injections

$$\iota_A : A \rightarrow A + B, \quad \iota_B : B \rightarrow A + B,$$

such that for every $f : A \rightarrow X, g : B \rightarrow X$, there is exactly one arrow

$$[f, g] : A + B \rightarrow X$$

whose composites with the injections are f and g .

In typed programming, tagged unions are coproduct-like. SessionStream’s logical payload families have this shape:

$$\text{EventPayload} = E_1 + E_2 + \cdots + E_k.$$

A consumer of the coproduct is defined by handling every variant. Protobuf `oneof` is a concrete encoding of a finite sum. The current `Event` structure uses a string name plus a dynamic

`proto.Message`; the schema registry provides runtime evidence for the intended tagged-sum discipline.

An untagged `google.protobuf.Struct` weakens this structure because the alternatives and their eliminators are no longer explicit.

4.4 Equalizers

Definition. Given parallel arrows $f, g : A \rightarrow B$, an equalizer is an arrow

$$e : E \rightarrow A$$

such that $f \circ e = g \circ e$, and for every $h : X \rightarrow A$ satisfying $f \circ h = g \circ h$, there is exactly one $u : X \rightarrow E$ with $e \circ u = h$.

In **Set**,

$$E = \{a \in A : f(a) = g(a)\}.$$



Figure 4.2: An equalizer can select histories on which live and replay semantics agree.

4.4.1 Replay agreement as an equalizer

Let H be a set of histories and let

$$L, R : H \rightarrow V$$

be live materialization and rebuild materialization into an observable view space V . The equalizer

$$\text{Eq}(L, R) \hookrightarrow H$$

is the subspace of histories for which replay agrees with live behavior.

This is more precise than the sentence “replay should be deterministic.” It separates:

- the domain of histories under consideration;
- the observation used for equality;
- the two competing interpretations;
- the maximal subobject on which they agree.

A failing test supplies a history outside the equalizer.

4.5 Coequalizers

The dual of an equalizer identifies outputs that should be treated as equivalent.

In **Set**, the coequalizer of $f, g : A \rightarrow B$ is the quotient of B by the smallest equivalence relation forcing

$$f(a) \sim g(a)$$

for every $a \in A$.

Software quotienting appears when several representations are normalized to one semantic value. For example, if different accepted JSON encodings are declared equivalent, the semantic decode space can be seen as a quotient. A coequalizer describes the universal quotient that performs the required identifications and no more.

Quotients are dangerous when operational evidence matters. If two traces end in the same state but differ in drops, timeouts, or authorization decisions, coequalizing them may erase the property you intended to verify.

4.6 Diagrams and cones

A diagram D in $\mathcal{C}$ is a collection of objects and arrows with a specified shape. Formally, a diagram is a functor

$$D : J \rightarrow \mathcal{C}$$

from an indexing category J , although Goldblatt first introduces it less formally.

A cone from an object X to D assigns an arrow

$$\lambda_j : X \rightarrow D(j)$$

for every object j of the diagram, compatible with every arrow of the diagram. If $u : i \rightarrow j$ is in J , then

$$D(u) \circ \lambda_i = \lambda_j.$$

The cone is one candidate global observation whose projections agree with the diagram's internal relationships.

4.7 Limits

Definition. A limit of $D : J \rightarrow \mathcal{C}$ is a cone (L, λ_j) such that for every cone (X, x_j) , there is exactly one arrow $u : X \rightarrow L$ satisfying

$$\lambda_j \circ u = x_j$$

for every j .

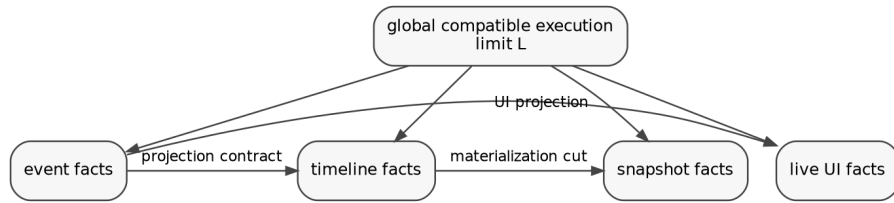


Figure 4.3: A limit as the universal compatible execution.

A limit is the universal compatible assembly of the diagram's pieces.

This single definition includes:

- terminal objects: limits of empty diagrams;
- products: limits of discrete two-object diagrams;
- equalizers: limits of parallel-arrow diagrams;
- pullbacks: limits of cospans.

This is why §3.11 is a conceptual hinge. Products and equalizers are not isolated tricks. They are instances of one construction.

4.7.1 Limits as compatible databases of facts

Suppose a diagram contains:

- event facts E ;
- timeline facts T ;
- snapshot facts S ;
- live UI facts U ;

with arrows expressing the projection and cut contracts. A point of the limit is a tuple

$$(e, t, s, u)$$

whose components satisfy every compatibility equation.

In **Set**, limits can often be constructed as subsets of products:

$$L \subseteq E \times T \times S \times U$$

containing exactly the compatible tuples.

This is the categorical version of a constrained join. The universal property says any other system supplying compatible component facts factors uniquely through this one.

4.8 Colimits

A cocone reverses the cone arrows, and a colimit is universal among cocones. Colimits assemble by identification rather than by compatibility constraints.

Examples include:

- initial objects: colimits of empty diagrams;
- coproducts: colimits of discrete diagrams;
- coequalizers: colimits of parallel arrows;
- pushouts: colimits of spans.

A rough engineering mnemonic is:

- limits select tuples that agree;
- colimits merge pieces while imposing identifications.

The mnemonic is reliable in **Set**, but the universal property is the definition.

4.9 Pullbacks

Definition. Given arrows $f : A \rightarrow C$ and $g : B \rightarrow C$, a pullback is an object P with arrows $p_A : P \rightarrow A$ and $p_B : P \rightarrow B$ such that

$$f \circ p_A = g \circ p_B,$$

and universal among all such commuting pairs.

In **Set**,

$$P = A \times_C B = \{(a, b) : f(a) = g(b)\}.$$

4.9.1 Schema validation as a pullback

Let:

- N be logical event names;
- P be protobuf payload values;
- D be protobuf descriptors;

- $r : N \rightarrow D$ be registry lookup;
- $t : P \rightarrow D$ be runtime reflection.

Then validated named payloads form the pullback

$$V = N \times_D P = \{(n, p) : r(n) = t(p)\}.$$

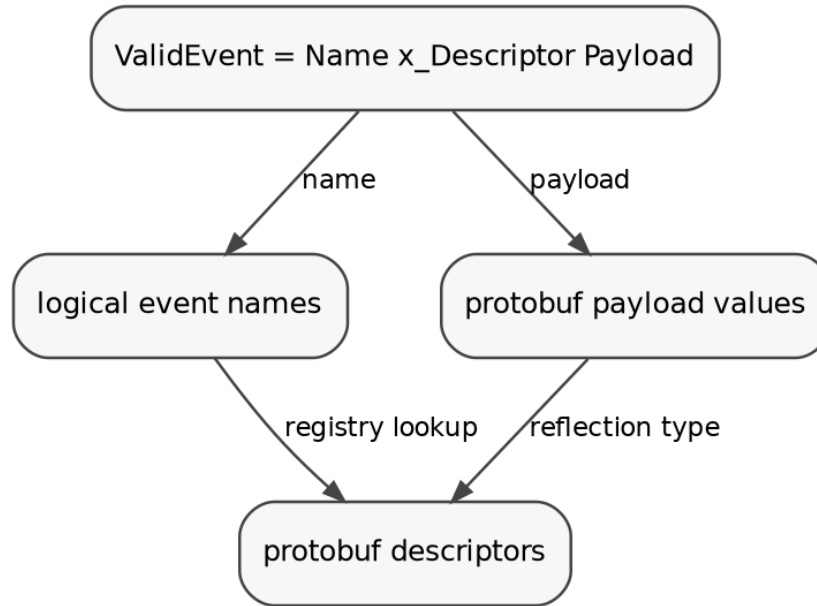


Figure 4.4: Schema validation as a pullback.

The concrete `SchemaRegistry` and `Hub.validatePayloadType` implement this matching idea: the name selects a prototype descriptor, the payload supplies its actual descriptor, and validation requires equality.

The pullback is stronger than saying “check the two descriptors.” It is the canonical object of all matching pairs. Any other typed event representation carrying a name and payload that agree over descriptors factors uniquely through it.

4.9.2 Reindexing by pullback

Suppose $f : A \rightarrow B$ maps detailed scopes to coarser scopes. A bundle of data over B can be pulled back along f to a bundle over A . In database language this resembles reindexing or joining data with the scope map. In topos theory, pullback becomes the central reindexing operation, and its adjoints become quantifiers.

4.10 Pushouts

A pushout is the dual of a pullback. Given $C \rightarrow A$ and $C \rightarrow B$, it merges A and B while identifying the two images of C .

A protocol migration can sometimes be modeled this way. An old schema and a new schema may share a stable common core. Their pushout is a universal merged representation in which the two copies of the core are identified.

This is an abstract construction, not an automatic recommendation. Real migrations must preserve semantics, compatibility, and operational constraints that may not be represented by the chosen category.

4.11 Finite completeness

A category is finitely complete when every finite diagram has a limit. It is enough to have a terminal object and pullbacks. From those, finite products and equalizers can be constructed.

For software reasoning, finite completeness means the modeling universe can express finite systems of compatible observations. You can:

- add contextual variables through products;
- impose equations through equalizers;
- reindex dependent data through pullbacks;
- combine these operations coherently.

An elementary topos will be finitely complete. This supplies the structural substrate for predicates and substitution.

4.12 Exponentials

Definition. An exponential B^A is an object with an evaluation arrow

$$\text{ev} : B^A \times A \rightarrow B$$

such that for every $f : X \times A \rightarrow B$, there is exactly one arrow

$$\hat{f} : X \rightarrow B^A$$

satisfying

$$\text{ev} \circ (\hat{f} \times 1_A) = f.$$

In **Set**, B^A is the set of all functions from A to B . The universal property is currying:

$$\text{Set}(X \times A, B) \cong \text{Set}(X, B^A).$$

A category with finite products and exponentials is Cartesian closed.

4.12.1 Handler and projection spaces

Mathematically, the set of pure event interpreters $S \times E \rightarrow S$ can be represented as an exponential

$$S^{S \times E}.$$

A configuration-dependent projection

$$C \times (S \times E) \rightarrow S$$

can be curried to

$$C \rightarrow S^{S \times E}.$$

This says a configuration selects a projector. It gives a clean model for dependency injection when dependencies are explicit values.

Actual Go functions are not automatically elements of a set with usable equality, but the mathematical semantics can still be Cartesian closed.

4.13 A complete worked limit: coherent hydration

Fix a session s and a target cut m . Consider these local facts:

1. a snapshot S_n at cut $n \leq m$;
2. a suffix $q = e_{n+1} \cdots e_m$;
3. a reconstructed client state C_m ;
4. a timeline fold result T_m .

Compatibility equations are:

$$\text{fold}(S_n, q) = C_m,$$

$$\text{materialize}(e_1 \cdots e_m) = T_m,$$

$$\text{clientView}(T_m) = C_m,$$

and the suffix ordinals are strictly greater than n and ordered.

The set of coherent hydration witnesses is the limit of this constraint diagram. A witness is not just a client state. It is the entire tuple with all compatibility evidence.

This distinction is useful in testing. A test that checks only final client state projects the limit witness onto one component. A stronger test records the cut, suffix, timeline state, and delivery trace, then verifies all equations.

4.14 When limits fail to exist in the model

A category may lack a desired limit, or a specific family of observations may have no compatible tuple.

These are different failures.

- **Missing construction:** the category does not support the required universal object.
- **Empty instance:** the limit object exists, but this particular constraint fiber has no element.
- **Non-unique assembly:** compatible local data have several global completions; the presheaf will later fail the uniqueness part of sheaf gluing.
- **Model mismatch:** the implementation uses hidden coordinates not present in the diagram.

For example, cursor and entity observations can each be locally valid while no consistent-cut snapshot contains both. That is an empty compatibility fiber for the claimed cut.

4.15 Exercises

4.1 [proof]. Prove that products are unique up to unique isomorphism.

4.2 [calculation]. In the prefix poset of histories, compute the product and coproduct of two histories when they exist. Interpret the result as greatest common prefix and least common extension.

4.3 [proof]. Show that an equalizer arrow is monic.

4.4 [design]. Define two arrows from event histories to an observable client model: one for the live path and one for rebuild. State a practical method for sampling their equalizer.

4.5 [calculation]. For registry map $r : N \rightarrow D$ and payload-type map $t : P \rightarrow D$, list the elements of the pullback for a toy registry with two names and three payloads.

4.6 [proof]. Show that a pullback in **Set** satisfies the universal property, not only the matching-pair equation.

4.7 [design]. Model authorization as a pullback or explain why a pullback is insufficient. Distinguish type admission from authority.

4.8 [proof]. Show that a terminal object is the limit of the empty diagram and a product is the limit of a discrete two-object diagram.

4.9 [calculation]. Construct the limit in **Set** of a diagram $A \xrightarrow{f} C \xleftarrow{g} B$. Then add a fourth set D with maps into A and B that commute over C . Write the unique mediating function.

4.10 [lab]. Build a small property test for snapshot-plus-suffix reconstruction. Generate an event history, choose a cut, materialize the prefix, replay the suffix, and compare with the full fold. Record enough data to diagnose which compatibility equation failed.

4.11 [research]. Draw a finite diagram for atomic projection progress containing event append, timeline apply, projection cursor, and fanout. Which subdiagram should be committed atomically? Which effect should remain outside the durable transaction?

4.12 [proof]. Derive the currying bijection in **Set** and verify both inverse equations.

4.13 [design]. Give one useful pushout-shaped schema migration and one case where a pushout would collapse distinctions that must remain separate.

4.14 [checkpoint]. Without notes, define cone, limit, pullback, and exponential. For each, state the universal quantification over competing candidates and the uniqueness clause.

Chapter 5

Subobjects, Classifiers, and the Topos Idea

Source alignment: Goldblatt, Chapter 4: subobjects, classifying arrows, the definition of an elementary topos, first examples, bundles and sheaves, monoid actions, power objects, and comprehension.

5.1 Subobjects are embeddings up to representation

A subset $A \subseteq B$ can be represented by its inclusion function

$$i : A \hookrightarrow B.$$

The categorical generalization is a monic arrow into B .

Two monics

$$m : A \hookrightarrow B, \quad n : C \hookrightarrow B$$

represent the same subobject when there is an isomorphism $u : A \cong C$ with

$$n \circ u = m.$$

The domain's labels do not matter; the way it sits inside B does.

Definition. A subobject of B is an equivalence class of monic arrows with codomain B , under isomorphism over B .

In **Set**, subobjects correspond exactly to subsets. In a general category they encode predicates, embedded structures, or admissible states without referring to literal membership.

5.1.1 Software invariants as subobjects

Let X be the set of all representable `SessionStream` runtime records of some type. An invariant P selects valid records

$$X_P = \{x \in X : P(x)\}.$$

The inclusion

$$X_P \hookrightarrow X$$

is the subobject corresponding to the invariant.

Examples include:

$$\text{SessionId} \neq "",$$

$$\text{LastEventOrdinal}(x) \leq \text{SnapshotOrdinal},$$

$$\text{payloadDescriptor}(p) = \text{registeredDescriptor}(n).$$

The subobject is not merely a Boolean check. It is the type of states carrying evidence that the check holds.

5.2 Generalized elements

In **Set**, an element $x \in X$ corresponds to a function

$$\bar{x} : 1 \rightarrow X$$

from a singleton. In a general category, arrows $1 \rightarrow X$ are called global elements.

But global elements may be too few to reveal an object. Category theory therefore uses generalized elements:

$$x : U \rightarrow X.$$

Here U is a context. You can read x as an X -valued quantity depending on variables in U .

This is a major bridge to software. A value is rarely observed without context. An event depends on a session and cut; a snapshot depends on a database state; a UI event depends on a subscrip-

tion. The arrow $U \rightarrow X$ makes the context explicit rather than pretending every value is globally available.

5.3 Characteristic functions in **Set**

For a subset $A \subseteq X$, define the characteristic function

$$\chi_A : X \rightarrow \{0, 1\}$$

by $\chi_A(x) = 1$ exactly when $x \in A$.

Let

$$\mathbf{true} : 1 \rightarrow \{0, 1\}$$

select 1. Then the inclusion $A \hookrightarrow X$ is the pullback of $\mathbf{true}$ along χ_A .

This pullback square says that A consists exactly of those elements of X classified as $\mathbf{true}$.

5.4 Subobject classifiers

Definition. A subobject classifier in a category $\mathcal{C}$ is an object Ω together with an arrow

$$\mathbf{true} : 1 \rightarrow \Omega$$

such that every monic $m : A \hookrightarrow X$ is, up to the standard equivalence, the pullback of $\mathbf{true}$ along a unique characteristic arrow

$$\chi_m : X \rightarrow \Omega.$$

The object Ω internalizes truth values. A predicate on X is represented by an arrow $X \rightarrow \Omega$.

In **Set**, $\Omega = \{0, 1\}$. In a presheaf or sheaf topos, Ω is richer: a proposition can have stage-dependent or local truth rather than one global Boolean value.

5.4.1 A classifier for typed admission

At a simple set level, let X be all pairs $(\mathbf{name}, \mathbf{payload})$. Let $A \subseteq X$ be pairs whose payload descriptor matches the registry. The characteristic map

$$\chi_A : X \rightarrow \{0, 1\}$$

is the validation predicate.

The pullback of `true` is the set of admitted pairs. In code, returning a refined type such as `ValidatedEvent` rather than a bare Boolean corresponds more closely to using the pullback object.

5.5 The definition of an elementary topos

Goldblatt first states the original Lawvere-Tierney definition and then notes the redundancy of finite cocompleteness.

Definition. An elementary topos is a Cartesian closed category with a subobject classifier.

Equivalently, in the expanded form used pedagogically:

- it has finite limits;
- it has exponentials;
- it has a subobject classifier.

Finite colimits follow from these conditions, although proving that is nontrivial.

A topos has enough Set-like structure to support:

- finite contexts and equations;
- function objects;
- predicates as arrows into Ω ;
- an internal intuitionistic logic;
- power objects and comprehension.

A topos is not simply “a category with topology.” The word has historical roots in Grothendieck’s sheaf theory, but elementary topoi include `Set`, finite sets, presheaf categories, and many other universes.

5.6 Power objects

In `Set`, the power set $\mathcal{P}(A)$ classifies subsets of A . Categorically, a power object PA classifies subobjects of $A \times X$ naturally in X .

In a Cartesian closed category with subobject classifier,

$$PA = \Omega^A.$$

An element of Ω^A is a predicate on A . The evaluation map

$$\Omega^A \times A \rightarrow \Omega$$

is membership.

For software, Ω^A is the space of predicates or policies on A , at the mathematical level. A concrete policy engine usually represents only a computable or syntactically describable subset of all predicates.

5.7 Topos thinking for API contracts

A useful sequence is:

1. choose an object X of candidate requests or states;
2. represent each invariant as a subobject $A \hookrightarrow X$;
3. obtain its characteristic arrow $\chi_A : X \rightarrow \Omega$;
4. combine predicates using the internal algebra of Ω ;
5. pull predicates back along API or projection maps;
6. quantify along context maps using adjoints, when available.

This reframes “validation” as structural predicate manipulation. It also reveals why parameter sufficiency is a pullback/reindexing question: a predicate on full states can be pulled back to supplied parameters only if it is constant across the omitted fibers, or if the API supplies enough evidence to decide it.

5.8 First encounter with sheaves

Goldblatt introduces sheaves of germs in Chapter 4 before returning to presheaves and sheaves of sections in Chapter 14. This companion follows the same spiral.

At this stage, retain only the picture:

- a base space supplies locations or contexts;
- over each location there is a stalk of possible local values;
- a section chooses values continuously or compatibly across a region;
- the category of sheaves behaves like a universe of variable sets.

For `SessionStream`, an eventual analogue will take contexts to be observation regions rather than physical opens. Sections will be compatible assignments of events, cuts, projections, and client states.

5.9 Exercises

5.1 [proof]. Verify that equivalence of monics over X is an equivalence relation.

5.2 [design]. Define the subobject of `Snapshot` values satisfying consistent-cut safety. What evidence would a refined `ConsistentSnapshot` type carry?

5.3 [calculation]. For $X = \{0, 1, 2, 3\}$ and $A = \{1, 3\}$, write the characteristic map and the pullback square classifying A .

5.4 [proof]. Show that predicates $X \rightarrow \{0, 1\}$ correspond bijectively to subsets of X .

5.5 [design]. Give a reason to return a validated value rather than only `bool` or `error`. Relate the answer to the pullback of `true`.

5.6 [research]. List the parts of a real `SessionStream` implementation that prevent it from literally being an elementary topos. Then identify a mathematical semantics that plausibly is one.

Chapter 6

Images, Reachability, and Extensionality

Source alignment: Goldblatt, Chapter 5: monics as equalizers, images, fundamental structural facts, extensionality, bivalence, and characterizations by elements.

6.1 Monics as equalizers

In a topos, every monic arrow can be exhibited as an equalizer. Intuitively, every subobject is the locus where two characteristic behaviors agree.

For a monic $m : A \hookrightarrow X$ with characteristic arrow $\chi_m : X \rightarrow \Omega$, the inclusion is the equalizer of χ_m and the constant-true predicate:

$$A \hookrightarrow X \rightrightarrows \Omega.$$

This connects predicates and equations. To satisfy a predicate is to equalize its truth value with `true`.

In software terms, a refined type can be described either by an admission predicate or by an equalizer of two observations.

6.2 Images

For a function $f : A \rightarrow B$, its image is the subset of outputs actually reached. Categorically, an image factorization has the form

$$A \twoheadrightarrow \text{Im}(f) \hookrightarrow B.$$

The first arrow is epic and the second monic. In a topos, such factorizations behave well.

6.2.1 Reachable materialized states

Let

$$\text{fold} : E_s^* \rightarrow S_s$$

map histories to timeline states. The image

$$\text{Reach}_s \hookrightarrow S_s$$

is the subobject of reachable states.

A state can satisfy all local field validations and still lie outside this image. Reachability is a global historical invariant, not a record-level schema invariant.

This distinction matters in repair tooling. Directly editing rows may create an entity configuration that cannot arise from any valid event history.

6.3 Image as the smallest containing subobject

The image of $f : A \rightarrow B$ is the smallest subobject of B through which f factors. This universal property is more useful than “the set of outputs” because it generalizes beyond **Set**.

For an event handler’s published-event map, the image tells you which canonical events are reachable from admitted commands under the modeled handler semantics. If a schema registers events outside this image, they may still be produced by other handlers or recovery processes; the image is always relative to the chosen arrow.

6.4 Extensionality

Extensionality says that arrows are determined by their behavior on arguments. In **Set**, if

$$f \circ x = g \circ x$$

for every element $x : 1 \rightarrow A$, then $f = g$.

In an arbitrary category, global elements may not be enough to distinguish arrows. Generalized elements often are: the Yoneda perspective says an arrow is determined by all its composites from all contexts.

For software, this warns against testing only global or default contexts. Two projections can agree on every current fixture and differ on a context not represented in the test suite.

A robust extensional test scheme quantifies over:

- session metadata;
- prior timeline views;

- schema versions;
- event prefixes;
- cancellation states;
- transport timing states.

6.5 Bivalence and its limits

In **Set**, global truth values are the two elements of $\{0, 1\}$. Every global proposition is either true or false.

In a general topos, Ω can have more global or local structure, and excluded middle may fail internally. This does not mean the external metatheory has abandoned ordinary truth. It means the internal language of the modeled universe tracks evidence or locality differently.

For a partially observed run, the proposition “this run finishes successfully” may not yet have a proof, nor may its negation have a proof. The runtime’s eventual outcome is classically determined in a fixed complete execution, but the stage-indexed information model is intuitionistic.

6.6 Observable equivalence

Let two implementations $p, q : I \rightarrow O$ be compared through an observation map $o : O \rightarrow V$. They are observationally equal when

$$o \circ p = o \circ q.$$

This is weaker than $p = q$. It may intentionally forget logs, allocation, timing, or internal entity order.

The equalizer of $o \circ p$ and $o \circ q$ identifies inputs on which the implementations are observationally equivalent. Changing o changes the theorem.

The Architecture Garden’s law

$$\text{observe}(\text{concurrentApply}(H_s)) = \text{observe}(\text{fold}(H_s))$$

is explicitly an observational equation. The choice of **observe** is part of the contract.

6.7 Exercises

6.1 [proof]. Show in **Set** that the inclusion of a subset $A \subseteq X$ equalizes its characteristic function and the constant-true function, and satisfies the universal property.

6.2 [design]. Define three different observation maps for comparing live and replayed Session-Stream state: storage-level, client-level, and audit-level. Give a pair of implementations equal under one and unequal under another.

6.3 [calculation]. For a toy event fold, enumerate its reachable states and construct the image inclusion.

6.4 [research]. Identify a direct database edit that preserves row schemas but leaves the image of valid event-history materialization.

6.5 [proof]. Explain why testing arrows only on global elements can fail in a category with too few global elements. Use generalized elements to state a stronger criterion.

Chapter 7

Logic as Algebra of Invariants

Source alignment: Goldblatt, Chapters 6-7: classical propositional logic, Boolean algebra, truth functions as arrows, and the lattice of subobjects.

7.1 Propositions as subobjects

A proposition in a topos can be represented as a subobject of the terminal object:

$$P \hookrightarrow 1.$$

In **Set**, there are only two such subobjects up to equivalence: the empty subset and the singleton itself. Hence global propositions have two truth values.

A predicate with a free variable of type X is a subobject

$$P \hookrightarrow X$$

or equivalently a characteristic arrow

$$\chi_P : X \rightarrow \Omega.$$

This is the categorical form of a typed invariant.

7.2 Conjunction and intersection

Given subobjects $P, Q \hookrightarrow X$, their conjunction is their intersection, constructed as a pullback:

$$P \wedge Q = P \times_X Q.$$

A state satisfies the conjunction exactly when it factors through both subobjects.

For a snapshot, consider:

$P(x)$: all entity ordinals are at most the snapshot cut,

$Q(x)$: entity keys are unique.

The valid-snapshot object for both conditions is the pullback intersection.

7.3 Disjunction and union

In **Set**, disjunction corresponds to union. In a topos, unions of subobjects can be constructed using images of coproduct maps:

$$P + Q \rightarrow X.$$

The coproduct remembers which proof branch supplied membership; taking the image forgets the tag and retains the union subobject.

This proof-sensitive distinction is familiar in typed programming. A value of `Either[P,Q]` carries evidence of which branch holds. A Boolean `P || Q` may erase that evidence.

7.4 Negation

Classically, $\neg P$ is the complement of P . In intuitionistic logic, negation means

$$P \Rightarrow \perp.$$

It says that a proof of P would lead to contradiction. It does not automatically produce a decidable complement.

For runtime information, “not observed to have finished” is not the same as “observed to be unfinished forever.” Confusing these is a common temporal-logic bug.

7.5 Implication

In a Heyting algebra of subobjects, implication $P \Rightarrow Q$ is the largest predicate R such that

$$R \wedge P \leq Q.$$

This is an adjoint property:

$$R \wedge P \leq Q \iff R \leq (P \Rightarrow Q).$$

The meaning is operationally useful. $P \Rightarrow Q$ is the weakest additional condition under which P guarantees Q .

For example:

- P : a subscription has been registered;
- Q : every post-registration batch is represented or delivered;
- R : buffering remains below capacity and snapshot transition completes.

Then R can be studied as a sufficient condition for the desired implication.

7.6 The subobject lattice

For each object X , its subobjects form an ordered structure

$$\text{Sub}(X),$$

where $P \leq Q$ means P factors through Q , corresponding to logical implication.

In a topos, $\text{Sub}(X)$ is a Heyting algebra. It supports finite meets, joins, implication, top, and bottom. It need not be Boolean.

This makes invariant composition algebraic. One can ask:

- Which invariant is stronger?
- What is their conjunction?
- What is the least invariant implied by either?
- What additional assumption makes one imply another?

7.7 Pulling invariants backward

Given $f : X \rightarrow Y$ and a predicate $Q \hookrightarrow Y$, its pullback

$$f^{-1}(Q) \hookrightarrow X$$

is the predicate “ $f(x)$ satisfies Q .”

This is substitution. If f is an API-to-domain map, pulling a domain invariant backward gives the request-level condition that guarantees it.

For parameter sufficiency, let $r : X \rightarrow P$ forget the omitted domain coordinates and retain supplied parameters. A full-state invariant $I \hookrightarrow X$ descends to a predicate on P only when it is constant in the relevant way over fibers of r . Otherwise no request-only Boolean can decide the invariant without additional lookup or evidence.

7.8 Classical logic as a special case

A topos is Boolean when every subobject has a complement, or equivalently when its internal logic validates excluded middle. **Set** is Boolean. General sheaf and presheaf topoi are often not.

Classical logic remains available externally while we reason about a non-Boolean internal universe. This two-level discipline matters:

- externally, we can prove a theorem about all stages;
- internally, a stage may lack evidence for $P \vee \neg P$.

7.9 Exercises

7.1 [calculation]. Construct the intersection of two subsets as a pullback and verify the universal property.

7.2 [design]. Express the SessionStream snapshot safety contract as a conjunction of at least four subobjects.

7.3 [proof]. In a Boolean algebra, show that $P \Rightarrow Q = \neg P \vee Q$. Explain why this identity is not the definition in a Heyting algebra.

7.4 [design]. Given a request parameter map $r : X \rightarrow P$, formulate a test for whether an invariant $I : X \rightarrow \{0, 1\}$ is determined by parameters alone.

7.5 [research]. Compare a proof-carrying coproduct **Either**[P,Q] with a Boolean disjunction. Which information is lost by taking the image/union?

Chapter 8

Intuitionistic Logic as Growing Information

Source alignment: Goldblatt, Chapter 8: constructivist motivation, Heyting calculus and algebras, Kripke semantics, and Beth-style local semantics.

8.1 Truth at a stage

Let P be the prefix poset of a session's histories. Interpret

$$h \Vdash \varphi$$

as “the information available at history h supports φ .”

Information grows along prefix extension. Therefore forcing should be monotone:

$$h \preceq k \text{ and } h \Vdash \varphi \implies k \Vdash \varphi.$$

Once a durable fact is established, later history should not invalidate the fact. This applies only to propositions modeled as persistent. “The current text is empty” is not persistent; “an inference-start event occurred” is.

8.2 Kripke semantics

For a preorder of stages, intuitionistic connectives can be interpreted as follows.

$$h \Vdash \varphi \wedge \psi$$

when both are forced at h .

$$h \Vdash \varphi \vee \psi$$

when one branch is forced at h with evidence of which.

$$h \Vdash \varphi \Rightarrow \psi$$

when for every extension $k \succeq h$, if $k \Vdash \varphi$, then $k \Vdash \psi$.

$$h \Vdash \neg\varphi$$

when no extension $k \succeq h$ forces φ .

The implication clause explains why implication is future-looking. To know $\varphi \Rightarrow \psi$ now, the guarantee must survive every refinement of information.

8.3 Why excluded middle can fail

Let F mean “the current inference eventually finishes normally.” At a prefix before a terminal event, neither F nor $\neg F$ may be forced.

- There is not yet evidence of normal finish.
- There may be an extension with normal finish, so negation is not supported.

Thus

$$F \vee \neg F$$

need not be forced at that stage.

Externally, each complete execution may have a definite outcome. The internal stage semantics is about available evidence, not metaphysical indeterminacy.

8.4 Heyting algebras of information

Open sets of a topological space form a Heyting algebra. So do upward-closed sets of stages in a Kripke frame.

A proposition corresponds to the set of stages where it is forced. Persistence makes this set upward closed.

Conjunction is intersection. Disjunction is union. Implication is

$$U \Rightarrow V = \{p : \forall q \succeq p, q \in U \Rightarrow q \in V\}.$$

Negation is $U \Rightarrow \emptyset$.

This gives a geometric picture: a proposition is a region of information space. Implication collects the points from which every future entrance into U also lies in V .

8.5 Safety and liveness

Intuitionistic stage semantics naturally distinguishes safety evidence from eventuality.

A safety property such as

no delivered live ordinal is at most the snapshot cut

can often be refuted by a finite bad prefix. Its negation may become forced once the bad delivery occurs.

A liveness property such as

every accepted observation is eventually delivered or explicitly dropped

cannot generally be established from a finite prefix without additional fairness or termination evidence.

This is not itself temporal logic, but the evidence-sensitive reading prevents classical shortcuts that confuse “not yet” with “never.”

8.6 Local proof objects

Constructive logic treats a proof of disjunction as a choice of branch and a proof of existential quantification as a witness. Software frequently needs exactly that information.

Instead of returning

`bool`

for “the batch is covered,” return a sum type:

```
CoveredBySnapshot(snapshotOrdinal)
| DeliveredInSuffix(eventOrdinal, deliveryId)
| ExplicitOverflow(reason)
```

This is a constructive proof object for the snapshot-plus-suffix completeness law. It is more useful for debugging and recovery than a Boolean summary.

8.7 Refinement of knowledge versus mutation of truth

A subtle but essential distinction:

- **knowledge refinement:** later contexts reveal more about one fixed execution;
- **world mutation:** later events change the system state.

Kripke semantics models monotone information, not arbitrary mutable predicates. To use event prefixes as stages, propositions must be chosen so that evidence persists along extension, or their time index must be included explicitly.

For example, “entity x has payload p at the current cut” is not persistent. The proposition

entity ‘ x ’ had payload ‘ p ’ at cut n

is persistent once cut n is included in history.

Adding coordinates turns mutable claims into stable historical claims.

8.8 Exercises

8.1 [calculation]. Build a four-stage prefix poset for a run that starts, emits a delta, and either finishes or stops. Compute the upward-closed set for “start occurred” and for “normal finish occurred.”

8.2 [proof]. Verify the monotonicity of the Kripke implication clause.

8.3 [calculation]. Give a stage at which neither F nor $\neg F$ is forced. Explain why this refutes stagewise excluded middle.

8.4 [design]. Rewrite a Boolean SessionStream status check as a constructive sum of evidence variants.

8.5 [research]. Choose five runtime propositions and classify them as persistent, nonpersistent, or persistent after adding an explicit cut/time coordinate.

8.6 [proof]. In the Heyting algebra of upward-closed subsets, derive the formula for implication and prove its adjunction with intersection.

Chapter 9

Functors, Natural Transformations, and Multiple Interpreters

Source alignment: Goldblatt, Chapter 9: functors, contravariant functors, natural transformations, equivalence, and functor categories.

9.1 Functors preserve categorical structure

Definition. A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ assigns an object $F(A)$ to each object A , and an arrow $F(f) : F(A) \rightarrow F(B)$ to each arrow $f : A \rightarrow B$, such that

$$F(1_A) = 1_{F(A)}$$

and

$$F(g \circ f) = F(g) \circ F(f).$$

A functor is an interpretation that respects doing nothing and doing things in sequence.

This is the formal core of “one architecture, several views.” If canonical histories form a category and each view interprets histories compositionally, then each view is functorial.

9.2 Histories acting on states

Treat E_s^* as a one-object category. Let $\mathbf{End}(S)$ be the one-object category whose arrows are state endomorphisms $S \rightarrow S$, with composition.

A deterministic event semantics assigns to each event e a state transformer

$$\delta_e : S \rightarrow S.$$

Extending by

$$F(e_1 \cdots e_n) = \delta_{e_n} \circ \cdots \circ \delta_{e_1}$$

and $F(\epsilon) = 1_S$ gives a functor

$$F : E_s^* \rightarrow \text{End}(S).$$

Functoriality is exactly the fold law. Concatenated histories are interpreted as composed state transformations.

The same event signature can have several functors:

$$F_T : E_s^* \rightarrow \text{End}(T)$$

for timeline state,

$$F_A : E_s^* \rightarrow \text{End}(A)$$

for audit state, and a writer-style interpretation for emitted UI words. This is the mathematically disciplined form of multiple interpreters.

9.3 Covariant and contravariant behavior

A contravariant functor $F : \mathcal{C}^{op} \rightarrow \mathcal{D}$ reverses arrows. If $U \subseteq V$, a presheaf supplies a restriction map

$$F(V) \rightarrow F(U).$$

More context has a map to less context because restriction forgets information.

Software contains both directions:

- execution extends forward from a shorter history to a longer one;
- observations restrict backward from a larger cut or context to a smaller one.

Confusing these directions is a common source of diagram errors. The base category arrow says which context is included in which; the presheaf arrow goes the opposite way.

9.4 Natural transformations

Definition. Given functors $F, G : \mathcal{C} \rightarrow \mathcal{D}$, a natural transformation $\eta : F \Rightarrow G$ assigns to each object A an arrow

$$\eta_A : F(A) \rightarrow G(A)$$

such that for every $f : A \rightarrow B$,

$$G(f) \circ \eta_A = \eta_B \circ F(f).$$

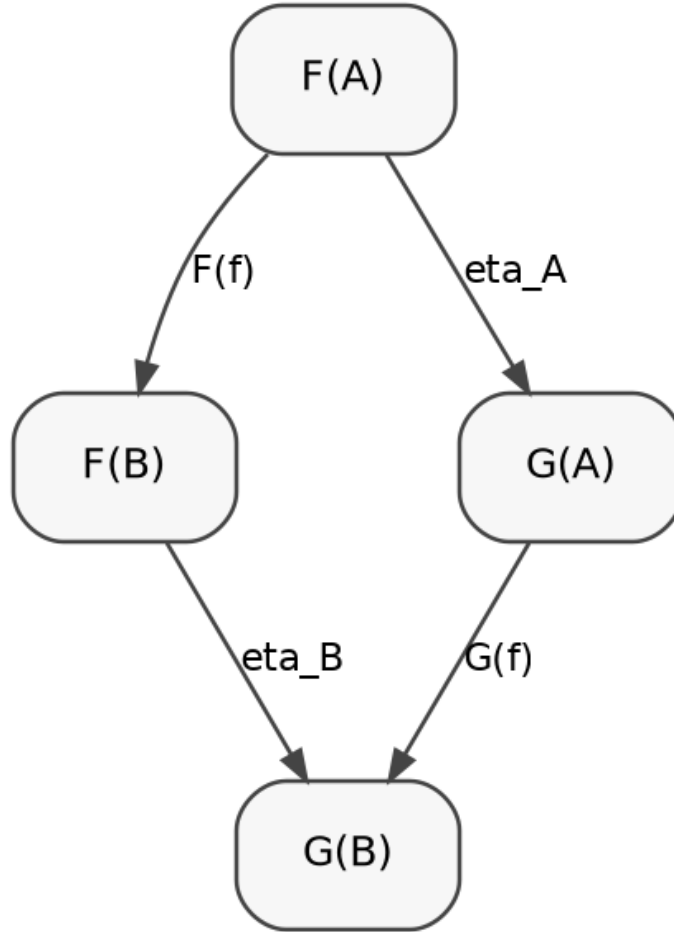


Figure 9.1: Naturality: translate then evolve equals evolve then translate.

The square says translation is uniform across structure.

9.4.1 Projection-version migration

Suppose F and G are old and new timeline semantics over the same history category, and $\eta_h : F(h) \rightarrow G(h)$ migrates the state at each history. Naturality says:

migrate after processing an extension = process the extension after migrating.

That is stronger than having a migration for each snapshot. It says migration commutes with every history arrow.

9.4.2 Encoding transformations

A family of codecs indexed by schema type is natural only if every schema-preserving transformation commutes with encoding. This is why code generation can be more principled than a collection of unrelated serializers: it can enforce a uniform translation.

9.5 Natural isomorphism and equivalence

A natural transformation is a natural isomorphism when each component is an isomorphism. Two functors connected by one are the same interpretation up to coherent representation change.

Categories $\mathcal{C}$ and $\mathcal{D}$ are equivalent when there are functors between them whose composites are naturally isomorphic to the respective identity functors.

Equivalence is often the correct notion for software representations. Requiring literal equality of type names or object identities is too strict; requiring only pairwise bijections is too weak because the bijections may not respect transformations.

9.6 Functor categories

For categories $\mathcal{C}$ and $\mathcal{D}$, the functor category $\mathcal{D}^{\mathcal{C}}$ has:

- functors $\mathcal{C} \rightarrow \mathcal{D}$ as objects;
- natural transformations as arrows.

Presheaves on $\mathcal{C}$ form the functor category

$$\mathbf{Set}^{\mathcal{C}^{op}}.$$

This is itself a topos when $\mathcal{C}$ is small. Thus a whole universe of context-dependent sets and restriction-preserving transformations arises from an ordinary category of contexts.

9.7 A caution about projections

A `SessionStream UIProjection` takes one event, session metadata, and a timeline view, and returns zero or more UI events. It is not automatically a functor as implemented. To obtain a functorial model, specify:

- a category of histories or transitions;
- an explicit state object containing all relevant prior information;
- composition by sequential event interpretation;
- equality of outputs, including whether UI word order matters.

The interface suggests an algebra, but the laws require proof or tests.

9.8 Exercises

9.1 [proof]. Extend an assignment of event generators to state endomorphisms into a unique functor from the free event-history category.

9.2 [design]. Give a naturality square for migrating timeline entity version 1 to version 2. State one likely failure of naturality.

9.3 [calculation]. For a two-object prefix category $0 \rightarrow 1$, list the data of a presheaf and the data of a natural transformation between two such presheaves.

9.4 [research]. Determine whether the current UI and timeline projections can be treated as two components of a product functor. List the purity and state assumptions required.

9.5 [proof]. Show that natural isomorphism is an equivalence relation on functors with fixed domain and codomain.

Chapter 10

Presheaf Topoi and Stage-Indexed Validity

Source alignment: Goldblatt, Chapter 10, where functor categories on preorders are used to analyze set concepts, the subobject classifier, truth arrows, and validity.

10.1 Variable sets

A presheaf is a set that varies over context. For a category $\mathcal{C}$, a presheaf

$$F : \mathcal{C}^{op} \rightarrow \mathbf{Set}$$

assigns:

- a set $F(U)$ of values available over each context U ;
- a restriction function $F(U) \rightarrow F(V)$ for each arrow $V \rightarrow U$;
- identity and composition laws for restriction.

A value is not simply “in F .” It is a section $s \in F(U)$ over some context U .

For event cuts, $F(n)$ might be the set of valid snapshots at cut n . Restriction from n to $m \leq n$ requires historical information; the current-only SQLite state does not by itself define this presheaf, while the entity-version table can support it.

10.2 Stage-indexed truth

In a presheaf topos over a preorder, the truth value of a proposition at a stage can be represented by the set of future refinements where it holds. Such sets are upward closed or, depending on arrow orientation, sieves.

The subobject classifier is therefore richer than $\{0, 1\}$. At context U , $\Omega(U)$ contains compatible collections of refinements of U .

This is a formal version of statements such as:

- true now and under every future extension;
- not established now, but established after particular refinements;
- locally true under a cover;
- impossible under every extension.

10.3 Subpresheaves

A subobject $A \hookrightarrow F$ in a presheaf category consists of subsets

$$A(U) \subseteq F(U)$$

closed under restriction. If a section satisfies the property over U , every restriction must satisfy the restricted property over smaller contexts.

This closure condition tests whether a proposed invariant is genuinely contextual.

For example, “this snapshot has no entity newer than its cut” is stable under a correct historical restriction. “This is the latest snapshot” is not stable under arbitrary restriction and needs a different index or formulation.

10.4 The Yoneda viewpoint

Every object C determines a representable presheaf

$$yC = \mathcal{C}(-, C).$$

At context U , $yC(U)$ is the set of arrows $U \rightarrow C$. The Yoneda lemma says

$$\text{Nat}(yC, F) \cong F(C).$$

A section at C is the same as a natural way of converting every generalized element of C into an F -section.

For software intuition, an object is completely characterized by how every context can map into it. This is the ultimate external-description principle.

Goldblatt does not center the early exposition on Yoneda, so this section is supplemental, but it clarifies why generalized elements are sufficient.

10.5 Parameter sufficiency as factorization

Let X be full semantic states and $r : X \rightarrow P$ retain API parameters. Let $I : X \rightarrow \Omega$ be an invariant predicate.

The parameters decide the invariant exactly when there exists a predicate

$$J : P \rightarrow \Omega$$

such that

$$I = J \circ r.$$

This is a factorization problem. In **Set** with Boolean truth, it means I is constant on every fiber

$$r^{-1}(p).$$

If two full states share the supplied parameters but give different invariant values, no parameter-only function can decide it.

The fiber formulation is:

$$\forall x_1, x_2, \quad r(x_1) = r(x_2) \implies I(x_1) = I(x_2).$$

For a richer presheaf of contexts, sufficiency must also be natural under restriction. A decision procedure that works only at one stage but not compatibly across refinements is not a presheaf morphism.

10.6 Context design is part of the theorem

Suppose `orderId` does not determine price because prices are versioned. The context object was too small. Replace it with `(orderId, priceVersion)`.

Mathematically, you refine the observation map

$$r : X \rightarrow P$$

to

$$r' : X \rightarrow P'.$$

The invariant may factor through r' even though it did not factor through r .

This provides a precise vocabulary for “missing parameter”: the current context identifies states that the invariant needs to distinguish.

10.7 Exercises

10.1 [design]. Define a cut-indexed presheaf of SessionStream event prefixes. Define every restriction map and verify the laws.

10.2 [design]. Define a presheaf of snapshots using historical `asOf` lookup. What happens at a cut for which no snapshot representation exists?

10.3 [proof]. Prove the fiber criterion for factorization of a Boolean invariant through a surjective parameter map.

10.4 [calculation]. Give a parameter map with fibers of sizes 1, 2, and 3. Define one invariant that factors through it and one that does not.

10.5 [research]. Choose a current REST or WebSocket request in your software. List all full-state coordinates relevant to authorization and invariants. Determine which are present, derivable, or missing.

Chapter 11

Internal Languages and Schemas

Source alignment: Goldblatt, Chapter 11: first-order language, semantics, models in a topos, substitution, soundness, Kripke models, completeness, partial elements, and higher-order logic.

11.1 Signatures before implementations

A first-order signature specifies:

- sorts;
- function symbols with input and output sorts;
- relation symbols with argument sorts.

A model interprets sorts as objects, function symbols as arrows, and relation symbols as subobjects.

A SessionStream schema registry resembles a fragment of a signature:

- command names and payload types;
- event names and payload types;
- UI event names and payload types;
- timeline entity kinds and payload types.

But a registry alone is not a logical theory. It does not state equations, invariants, quantifiers, or semantic truth conditions. It is admission metadata for typed alternatives.

11.2 A small SessionStream language

Consider sorts:

Session
Event
Ordinal
Entity
Snapshot

Function symbols might include:

$$\text{session} : \text{Event} \rightarrow \text{Session},$$

$$\text{ordinal} : \text{Event} \rightarrow \text{Ordinal},$$

$$\text{cut} : \text{Snapshot} \rightarrow \text{Ordinal},$$

$$\text{last} : \text{Entity} \rightarrow \text{Ordinal}.$$

Relations might include:

$$\text{ContainedIn}(\text{Entity}, \text{Snapshot}),$$

$$\text{ProjectsTo}(\text{Event}, \text{Entity}),$$

$$\text{DeliveredTo}(\text{Event}, \text{Connection}).$$

A consistent-cut axiom can then be written informally as

$$\forall x \forall s, \quad \text{ContainedIn}(x, s) \Rightarrow \text{last}(x) \leq \text{cut}(s).$$

A concrete model interprets these symbols in sets, database relations, traces, or internal objects of a topos.

11.3 Terms as arrows

A term with variables in context Γ and result sort A is interpreted as an arrow

$$\llbracket t \rrbracket : \llbracket \Gamma \rrbracket \rightarrow \llbracket A \rrbracket.$$

A context of variables is interpreted by a product. Substitution is composition.

If $t(x)$ is an ordinal-valued term and $u(y)$ supplies an event for x , then substituting u into t is

$$\llbracket t[u/x] \rrbracket = \llbracket t \rrbracket \circ \llbracket u \rrbracket.$$

This is why finite products and composition are prerequisites for logic.

11.4 Formulas as subobjects

A formula φ in context Γ is interpreted as a subobject

$$\llbracket \varphi \rrbracket \hookrightarrow \llbracket \Gamma \rrbracket.$$

Conjunction, disjunction, and implication use the Heyting algebra of subobjects. Substitution pulls the subobject back along the term arrow.

Logical soundness becomes a statement that derivable sequents correspond to subobject inclusions in every model.

11.5 Partial elements

Real software computations can fail to return a value. Goldblatt studies objects of partial elements later in Chapter 11. Categorically, a partial map can be represented by a domain subobject together with an arrow from that domain.

A partial decoder has:

$$D \hookrightarrow \text{Bytes}$$

as the subobject of decodable inputs and an arrow

$$D \rightarrow \text{Message}.$$

Returning **error** is one total encoding of this partiality. The domain-subobject model makes the admission predicate explicit.

For streaming handlers, termination is also partial at a finite stage. A terminal result exists only on the subobject of runs that have reached a terminal event.

11.6 Internal versus external claims

An internal statement is expressed in the language interpreted inside the category. An external statement is made in the surrounding mathematics about the category or model.

Example:

- internal: “every entity in this snapshot has `last <= cut`”;
- external: “the category of snapshots has a subobject interpreting that formula”;
- implementation: a SQL query and Go scan construct a value claimed to satisfy it.

Do not slide between these levels. A schema validator can establish a syntactic internal predicate without proving a business event’s external truth. A protobuf type proves shape, not authority or

historical accuracy.

11.7 Completeness is relative to a logic and model class

Goldblatt develops completeness results for logical systems and classes of models. In software, the analogous phrase “the checks are complete” is often used loosely.

A precise completeness claim must specify:

- the language of properties;
- the model class;
- the proof or checking system;
- whether every semantically valid property in that language is derivable or detected.

A linter that rejects top-level **Struct** is complete only for a narrowly specified syntactic policy, not for all schema quality.

11.8 Exercises

11.1 [design]. Write a many-sorted signature for `SessionStream` hydration. Include at least six sorts, eight functions, and six relations.

11.2 [proof]. Show how substitution of terms becomes composition of arrows.

11.3 [design]. Express snapshot-plus-suffix completeness as a family of first-order or temporal statements. Identify which parts exceed ordinary first-order logic.

11.4 [recognition]. Separate schema-level, internal logical, external semantic, and implementation claims for a validated **Event**.

11.5 [research]. State a narrow soundness and completeness theorem that `sessionstream-lint` could plausibly satisfy.

Chapter 12

Choice, Natural Numbers, and Recursive Folds

Source alignment: Goldblatt, Chapters 12-13: choice, natural numbers objects, categorical set theory, primitive recursion, and Peano-style structure.

12.1 The categorical axiom of choice

One categorical form of choice says every epic arrow splits: for every epic $e : A \rightarrow B$, there is $s : B \rightarrow A$ with

$$e \circ s = 1_B.$$

The section s chooses one preimage for each $b \in B$.

In **Set**, this is related to the ordinary axiom of choice. In a general topos it is a strong condition and can force classical logical behavior.

Software systems often make local choices through canonicalization, ordering, or IDs. Such mechanisms are explicit selection functions, not appeals to a global axiom. A deterministic retry selector, for example, needs an actual section with stability laws.

12.2 Natural numbers objects

Definition. A natural numbers object consists of an object N , a zero arrow

$$0 : 1 \rightarrow N,$$

and successor

$$s : N \rightarrow N,$$

satisfying a universal recursion property: for every object A , point $a : 1 \rightarrow A$, and endomorphism $f : A \rightarrow A$, there is a unique $h : N \rightarrow A$ with

$$h \circ 0 = a, \quad h \circ s = f \circ h.$$

The object is characterized by recursion, not by membership in a set-theoretic construction.

12.3 Ordinals are not automatically an NNO

`SessionStream` uses `uint64` ordinals. That implementation type is not itself a categorical natural numbers object in the runtime category. It is bounded, can overflow, and is interpreted through storage and transport conventions.

A mathematical model may use N as an NNO while proving that implementation ordinals refine a bounded prefix of it under an overflow precondition.

This separation is necessary for long-lived systems. A proof about N does not silently discharge `uint64` overflow.

12.4 Primitive recursion and folds

The NNO recursion property handles iteration indexed by natural numbers. Event replay is more naturally recursion over a free monoid or list object:

$$\text{fold}(S, \epsilon) = S,$$

$$\text{fold}(S, he) = \delta(\text{fold}(S, h), e).$$

The same universal theme appears: a recursive interpreter is the unique arrow satisfying base and step equations.

If events are indexed by ordinals, the history length and event ordinal may correspond, but they are conceptually distinct:

- length counts positions in a particular word;
- ordinal is an architecture-assigned coordinate;
- event identity names a logical occurrence;
- bus stream ID is a transport coordinate.

12.5 Induction as proof over construction

Induction proves a property for every value generated by zero and successor. For histories, structural induction proves:

1. the property holds for ϵ ;
2. if it holds for h , it holds for he .

Many SessionStream laws should be stated this way.

Example: if every event transition preserves entity-key uniqueness, and the empty state has unique keys, then every folded history has unique keys.

Concurrency can invalidate the sequential induction model. Then the theorem applies to the abstract serial fold, while a separate refinement theorem must show that concurrent execution is observationally equivalent to some legal serial order.

12.6 Recursion and snapshots

A snapshot can be viewed as a memoized fold result at cut n . The reconstruction law

$$\text{fold}(S_n, e_{n+1} \cdots e_m) = S_m$$

is a fusion or decomposition law for recursion.

A checkpoint is trustworthy when it is extensionally equal to folding the represented prefix. Otherwise suffix replay begins from a state outside the fold image and can compound the error.

12.7 Exercises

12.1 [proof]. Use structural induction on histories to prove the fold concatenation law.

12.2 [design]. State an invariant preserved by every timeline event transition. Write the base and induction steps.

12.3 [research]. List the proof obligations required to refine mathematical natural numbers to `uint64` ordinals.

12.4 [calculation]. For a toy state machine, compute folds by both left recursion and prefix checkpoint plus suffix. Verify equality.

12.5 [design]. Distinguish event length, event ordinal, stable event ID, and transport stream ID in types and laws.

Chapter 13

Presheaves as Contextual APIs

Source alignment: This chapter prepares Goldblatt Chapter 14 using the definitions of functor and limit already established. Goldblatt calls a presheaf over a topological space a “stack” in the older terminology used by the book.

13.1 The base category of contexts

A presheaf begins with a category of contexts $\mathcal{C}$. The choice of $\mathcal{C}$ determines what locality means.

For `SessionStream`, useful context coordinates include:

- session s ;
- event cut n ;
- observer or subsystem K ;
- schema version v ;
- connection or subscription scope c .

A context might therefore be a tuple

$$U = (s, n, K, v, c).$$

An arrow $V \rightarrow U$ should mean that V is a smaller, more local, or less informative context contained in U . Possible generators include:

- move from cut n to an earlier cut $m \leq n$;
- forget timeline facts while retaining event facts;
- forget connection-specific information;
- project a full schema to an older compatible interface;
- restrict a set of sessions to one session.

The direction must be chosen consistently. A presheaf will reverse these arrows.

13.2 Definition of a presheaf

Definition. A presheaf on $\mathcal{C}$ is a functor

$$F : \mathcal{C}^{op} \rightarrow \mathbf{Set}.$$

For each context U , $F(U)$ is the set of sections over U . For each arrow $i : V \rightarrow U$, there is a restriction map

$$F(i) : F(U) \rightarrow F(V), \quad s \mapsto s|_V.$$

Restrictions satisfy

$$s|_U = s,$$

and

$$(s|_V)|_W = s|_W$$

whenever $W \rightarrow V \rightarrow U$.

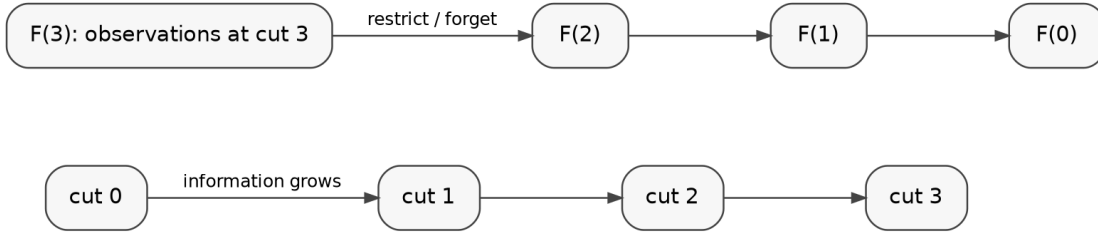


Figure 13.1: Information grows in the base direction while observations restrict contravariantly.

The word *section* should be read broadly: a local configuration, local observation, admissible assignment, or piece of data valid over the context.

13.3 A cut presheaf of histories

Fix a session s . Let the base category have natural cuts with an arrow $m \rightarrow n$ when $m \leq n$.

Define

$$H(n) = \{e_1 \cdots e_n\},$$

the singleton containing the actual prefix of one fixed execution. Restriction truncates a longer prefix to a shorter prefix.

This is a very simple presheaf: one section at each stage.

A more general presheaf has

$$\mathcal{H}(n) = \{\text{all admissible histories of length } n\},$$

with truncation. Now a global compatible family across all finite cuts is an infinite behavior whose every finite prefix is admissible. This begins to resemble safety-property semantics.

13.4 A snapshot presheaf

Let $S(n)$ be the set of coherent snapshots as of cut n . A restriction

$$S(n) \rightarrow S(m), \quad m \leq n,$$

must reconstruct the historical state at m .

The current SQLite design stores entity versions, so an `asOf` query can support this mathematically. A store retaining only current rows cannot generally define the restriction map. It may have values at each current cut, but no lawful way to restrict later state to earlier state.

This is a useful diagnostic:

Having stage-indexed values does not make a presheaf. You must provide compositional restriction maps.

13.5 A presheaf of observations

Let observer kinds be subsets of

$$\{E, T, S, U, C\}$$

for event, timeline, snapshot, live UI, and client observations. Order contexts by inclusion of visible dimensions.

Define $F(K)$ as the set of locally valid assignments to all coordinates visible in K . Restriction forgets coordinates.

For example,

$$F(\{E, T, S\}) \rightarrow F(\{T, S\})$$

forgets event details but retains timeline and snapshot facts.

This presheaf formalizes the architecture as overlapping partial views. It does not yet assert that locally valid views come from a global execution. That is the sheaf question.

13.6 Sections and fibers

Given a restriction $r : F(U) \rightarrow F(V)$ and a local section $v \in F(V)$, the fiber

$$r^{-1}(v)$$

contains all extensions of v to U .

For API parameters, V is the supplied parameter context and U is the full semantic context.

- Empty fiber: the request cannot be completed consistently.
- Singleton fiber: the parameters determine a unique full state.
- Multiple-element fiber: the request is underdetermined.
- Invariant-constant fiber: the full state is underdetermined, but the invariant has one value across all completions.

This is the cleanest elementary answer to “are these parameters enough?”

13.7 Presheaves of constraints

Let variables be distributed across contexts. Define

$$F(U) = \{\text{assignments to variables in } U \text{ satisfying constraints visible in } U\}.$$

Restriction forgets variables. This is a presheaf when restricting a valid assignment remains valid for the smaller set of visible constraints.

A global section is a full assignment satisfying every constraint.

This connects presheaves to constraint-satisfaction problems. It also supplies a practical software model before any additive algebra or cohomology is introduced.

13.8 When restriction is not mere field deletion

Restriction may require computation.

- Restricting a snapshot to an earlier cut requires historical lookup or replay.
- Restricting a transport trace to a subsystem requires projecting and possibly coalescing events.
- Restricting a schema to an older version may apply a migration.
- Restricting a materialized entity set to a tenant may enforce authorization.

The only requirements are functoriality:

$$\text{restrict}_{U \rightarrow U} = 1,$$

$$\text{restrict}_{U \rightarrow W} = \text{restrict}_{V \rightarrow W} \circ \text{restrict}_{U \rightarrow V}.$$

A migration chain that gives a different result from direct migration violates presheaf functoriality.

13.9 Presheaf morphisms

A natural transformation $\eta : F \Rightarrow G$ between presheaves is a context-compatible translation. For every restriction $V \rightarrow U$,

$$\eta_V(s|_V) = \eta_U(s)|_V.$$

Examples:

- translating internal timeline entities to client models at every cut;
- redacting observations while preserving restriction;
- migrating schema versions consistently across contexts;
- deriving metrics from traces so that metrics of a restricted trace equal restricted metrics.

A transformation that inspects unavailable global state is not natural with respect to the chosen contexts.

13.10 Exercises

13.1 [design]. Define a context category with coordinates (`session`, `cut`, `observer-kind`). Specify objects, generating arrows, and composition.

13.2 [proof]. Verify that prefix truncation defines a presheaf of admissible histories.

13.3 [design]. Determine whether current, historical, and replayable hydration stores define snapshot presheaves. Give the required restriction maps.

13.4 [calculation]. Construct fibers for a toy API parameter map and classify them as impossible, unique, underdetermined, or invariant-sufficient.

13.5 [proof]. Show that a family of schema migrations forms a presheaf restriction system exactly when identity and path-independence laws hold.

13.6 [research]. Define a presheaf of authorization evidence. What must restriction do when moving to a smaller scope?

Chapter 14

Sheaves, Covers, and Snapshot-Plus-Live Gluing

Source alignment: Goldblatt Chapter 14, especially §14.1 on presheaves and the compatibility condition COM, §§14.2-14.5 on classifying sheaves and sites, and §14.6 on Kripke-Joyal semantics.

14.1 Covers

A presheaf tells us how to restrict. A sheaf tells us when local pieces can be reconstructed globally.

On a topological space, a family of open subsets $\{U_i\}$ covers U when

$$U = \bigcup_i U_i.$$

In a general site, covering families are specified abstractly and must satisfy stability and transitivity laws. This lets “cover” mean collectively sufficient observation rather than literal spatial union.

For software contexts, possible covers include:

- snapshot prefix plus post-cut suffix;
- event log plus deterministic projector;
- several service responses whose fields jointly cover an invariant;
- replicas or shards that collectively cover a dataset;
- component traces that jointly cover an execution.

A cover is part of the modeling structure. Declaring a family to cover means you assert that its local information should suffice, subject to compatibility, to determine the global object.

14.2 Matching families

Let F be a presheaf and $\{U_i \rightarrow U\}$ a cover. Choose local sections

$$s_i \in F(U_i).$$

They form a matching family when they agree on every overlap:

$$s_i|_{U_i \cap U_j} = s_j|_{U_i \cap U_j}.$$

For a general site, overlaps are represented by pullbacks

$$U_i \times_U U_j.$$

The matching condition is pairwise equality after restriction to those pullbacks.

Pairwise compatibility is local evidence. It is necessary for gluing but, for a general presheaf, not sufficient.

14.3 The sheaf condition

Definition. A presheaf F is a sheaf for the chosen covers if every matching family $\{s_i\}$ has a unique amalgamation

$$s \in F(U)$$

such that $s|_{U_i} = s_i$ for every i .

Existence says compatible local data can be assembled. Uniqueness says the local data determine the global section.

Goldblatt's COM condition is precisely this compatibility-and-unique-pasting law.

The sheaf condition can also be expressed as a limit. For a cover, $F(U)$ is the equalizer of

$$\prod_i F(U_i) \rightrightarrows \prod_{i,j} F(U_i \cap U_j),$$

where the parallel arrows restrict each local section to the two sides of each overlap.

This equation directly links Chapter 4 limits to sheaves.

14.4 Snapshot plus live suffix as a cover

The WebSocket hydration protocol has two information regions:

$$U_{\text{past}} = \text{history represented through cut } n,$$

$$U_{\text{future}} = \text{accepted live observations after registration.}$$

The local sections are:

$$s_{\text{past}} = S_n,$$

$$s_{\text{future}} = q_{>n}.$$

Their overlap or boundary data includes at least:

- the same `SessionId`;
- the cut n ;
- an agreement that future batches have ordinals strictly greater than n ;
- ordering and duplicate semantics.

The desired amalgamation is the reconstructed client state

$$S_m = \text{fold}(S_n, q_{>n}).$$

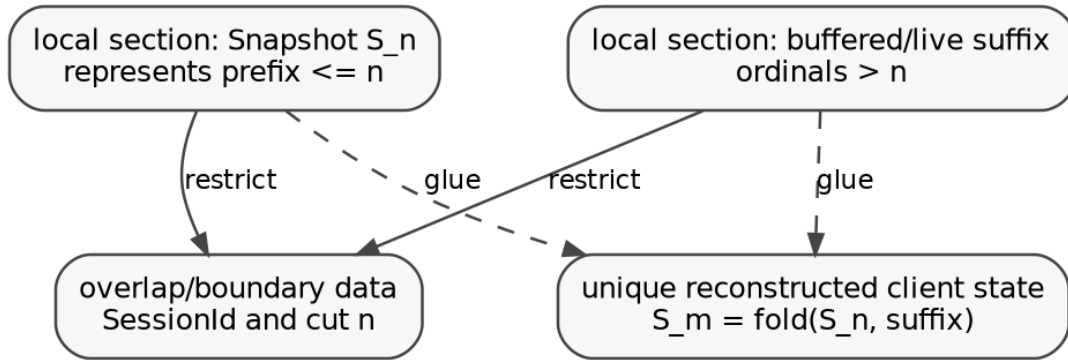


Figure 14.1: Snapshot and suffix as compatible local sections that glue into client state.

The current WebSocket implementation registers a subscription as hydrating before loading the snapshot, buffers concurrent UI batches, filters relative to `SnapshotOrdinal`, flushes later batches, and only then marks the subscription live. This is a concrete gluing algorithm.

14.5 Existence, uniqueness, and explicit failure

The sheaf condition has two independent parts.

14.5.1 Failure of existence

A snapshot says cut 42, but contains an entity with `LastEventOrdinal` = 43. No global “state at cut 42” restricts to both observations.

A hydration buffer overflows. The accepted local data cannot all be represented by the available snapshot-plus-buffer mechanism. Correct behavior is an explicit reconnect or overflow outcome, not silent gluing.

Two projections require incompatible values for one shared entity field. The matching family is not compatible.

14.5.2 Failure of uniqueness

Parameters omit the price version. Several full orders restrict to the same request but have different totals.

Two schema interpretations agree on every exposed field but differ on hidden semantics.

A partial trace omits the linearization point, allowing several global orderings.

Uniqueness failures correspond to underdetermination. Add coordinates, strengthen overlaps, or accept a quotient notion of global state.

14.6 The separatedness condition

A presheaf is *separated* when compatible local restrictions determine at most one global section. It satisfies uniqueness but not necessarily existence.

Software systems frequently achieve separatedness before full sheaf behavior:

- if a valid global execution exists, complete traces identify it uniquely;
- but some locally compatible traces may have no realizable global execution because of resource or timing constraints.

This distinction can sharpen architecture reviews. Ask separately:

1. Could two global states look identical through the cover?
2. Does every matching family correspond to a realizable state?

14.7 Sites and Grothendieck topologies

A site is a category equipped with a notion of covering. One common formulation uses covering sieves.

A **sieve** on U is a collection of arrows into U closed under precomposition. If $V \rightarrow U$ belongs to the sieve, then every composite $W \rightarrow V \rightarrow U$ also belongs.

A Grothendieck topology designates certain sieves as covering, subject to:

1. the maximal sieve covers;
2. covers are stable under pullback;
3. local covers compose to covers.

For software, these laws mean:

- complete observation is sufficient;
- sufficient observation remains sufficient after reindexing to a smaller context;
- if a family covers a context and each member is itself covered, the combined finer family covers the original.

This formalism lets you choose application-specific locality.

14.8 Example sites for SessionStream

14.8.1 Cut site

Objects are cuts. A family covers n when its represented prefixes and suffix intervals jointly cover all ordinals through n without gaps and with compatible boundaries.

14.8.2 Observer site

Objects are sets of observable dimensions. A family covers the global execution context when their union includes every coordinate required by the target invariant.

14.8.3 Transaction site

Objects are persistence facts. A family is covering only when the facts are read or committed under one transaction or a recovery protocol that supplies equivalent atomic evidence.

This last choice is important. Pairwise access to event append, entity state, and cursor does not automatically cover the atomic commit fact. The topology should not declare an invalid family sufficient.

14.9 Sheafification

Given a presheaf that fails gluing, one can often construct an associated sheaf by formally adding local equivalences and amalgamations. This is sheafification.

In engineering terms, sheafification resembles repairing an interface so that local data can be combined consistently. Possible operations include:

- add missing version coordinates;
- quotient representations by a justified local equivalence;
- insert a transaction boundary;
- retain enough history to define restrictions;
- add stable event IDs;

- replace implicit failure with an explicit outcome variant.

The analogy is useful, but mathematical sheafification is a specific universal construction: a left adjoint from presheaves to sheaves. Not every system repair is literally that construction.

14.10 Local truth and Kripke-Joyal semantics

Sheaf semantics interprets truth over a context U . A statement may hold locally on a cover without one global witness available on all of U .

The characteristic clauses include:

- conjunction holds when both conjuncts hold on U ;
- implication holds when it is preserved after every restriction;
- disjunction holds when a cover exists on whose pieces one branch or the other holds;
- existence holds when a cover exists with local witnesses;
- universal quantification holds after every restriction for every local element.

The disjunction and existential clauses are local. To establish

$$U \Vdash \exists x \varphi(x),$$

one may have witnesses x_i on covering regions U_i , without one global witness on U .

14.10.1 Software reading

A service federation may establish “there exists an owner for every shard” through local owners that do not glue to one global owner value. A UI may show one of two local rendering variants depending on feature context, even though no global branch is selected.

For SessionStream, “every accepted batch is covered” can be locally witnessed by either snapshot inclusion or suffix delivery. A constructive proof is a cover by these cases plus explicit overflow handling.

14.11 A detailed hydration matching condition

Let a hydration trace contain:

- registration time r ;
- loaded snapshot cut n ;
- set A of batches accepted after r ;
- set P represented by the snapshot;
- ordered delivered suffix D ;
- explicit failures F .

A completeness matching condition can be written

$$A = P \sqcup D \sqcup F$$

under a suitable identity relation, with

$$\forall d \in D, \quad \text{ord}(d) > n,$$

and delivery order respecting ordinal order.

The disjoint-union symbol emphasizes that every accepted batch has exactly one custody outcome. If duplicates are allowed, replace set equality with a multiset or trace relation.

A global section is a complete trace assigning every accepted batch to one lawful outcome while agreeing with snapshot and delivery facts on overlaps.

14.12 Why pairwise agreement may not be enough

Suppose three services expose pairwise compatible data:

- event store and timeline agree on cut;
- timeline and snapshot agree on entity rows;
- snapshot and client agree on rendered state.

There may still be no single event-to-client execution realizing all three if the pairwise agreements use different hidden versions. Pairwise overlaps omit a triple intersection coordinate.

This motivates higher-dimensional cells. An edge records pairwise compatibility. A filled triangle records a joint three-way witness. The nerve construction in Chapter 17 turns this overlap pattern into a shape.

14.13 Exercises

14.1 [proof]. Derive the equalizer formulation of the sheaf condition for a finite cover.

14.2 [design]. Specify the overlap object between a snapshot and a live suffix. Include identity, cut, ordering, duplicate, and failure data.

14.3 [lab]. Build a trace checker that partitions accepted hydration batches into snapshot-covered, suffix-delivered, and explicit-failure outcomes. Detect missing and duplicate custody.

14.4 [design]. Give one SessionStream presheaf that is separated but not a sheaf.

14.5 [proof]. Verify the Grothendieck-topology pullback-stability law for a simple cut-interval site.

14.6 [research]. Choose which families should cover the atomic projection-progress fact. Explain why separate nontransactional reads may fail to be a cover.

14.7 [calculation]. Construct a three-context example with pairwise compatible sections but no global section. Identify the missing triple-overlap information.

14.8 [research]. Describe a sheafification-like repair for stable retry identity. Which new coordinate is introduced, and how do restriction maps treat it?

Chapter 15

Adjunctions, Reindexing, and Quantifiers

Source alignment: Goldblatt, Chapter 15: adjunctions, universal arrows, products and exponentials as adjoint situations, the fundamental theorem of topoi, and quantifiers.

15.1 The hom-set correspondence

Definition. A functor $F : \mathcal{C} \rightarrow \mathcal{D}$ is left adjoint to $G : \mathcal{D} \rightarrow \mathcal{C}$, written

$$F \dashv G,$$

when there is a bijection

$$\mathcal{D}(F(A), B) \cong \mathcal{C}(A, G(B))$$

natural in A and B .

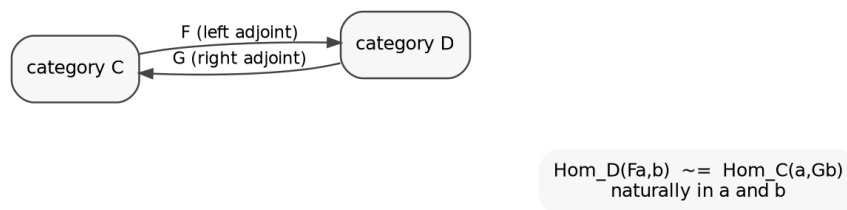


Figure 15.1: An adjunction is a natural correspondence between two kinds of arrows.

An adjunction says that two differently shaped design problems are equivalent in a coherent way.

15.2 Free and forgetful constructions

A classic pattern is:

free structure $\dashv$ forgetful functor.

A free monoid on an event alphabet E is the history set E^* . Any function from generators E to the underlying set of a monoid M extends uniquely to a monoid homomorphism

$$E^* \rightarrow M.$$

This is exactly why assigning semantics to event generators determines semantics for all histories.

In software, code generation often has a free/forgetful flavor: a schema declaration generates structured code, while forgetting returns the underlying signature. A literal adjunction requires carefully chosen categories and laws, so this should be treated as a modeling program rather than assumed.

15.3 Product and exponential adjunction

In a Cartesian closed category, product with A is left adjoint to exponentiation by A :

$$(-) \times A \dashv (-)^A.$$

The hom-set bijection is currying:

$$\mathcal{C}(X \times A, B) \cong \mathcal{C}(X, B^A).$$

For dependency-injected projectors, an arrow

$$C \times (S \times E) \rightarrow T$$

is equivalent to

$$C \rightarrow T^{S \times E}.$$

One view supplies configuration at each invocation; the other selects a configured projector once.

15.4 Reindexing predicates

Given $f : A \rightarrow B$, pullback sends a subobject of B to one of A :

$$f^* : \text{Sub}(B) \rightarrow \text{Sub}(A).$$

This is substitution or inverse image. A predicate on B becomes a predicate on A by applying f .

In **Set**, for $Q \subseteq B$,

$$f^*(Q) = f^{-1}(Q).$$

15.5 Existential quantification as a left adjoint

The left adjoint to inverse image is direct image:

$$\exists_f \dashv f^*.$$

For $P \subseteq A$,

$$\exists_f(P) = \{b \in B : \exists a \in P, f(a) = b\}.$$

The adjunction law is

$$\exists_f(P) \subseteq Q \iff P \subseteq f^{-1}(Q).$$

15.5.1 Software example

Let $f : \text{Event} \rightarrow \text{Session}$. For an event predicate P , $\exists_f(P)$ is the set of sessions having at least one event satisfying P .

For example:

$$\exists_f(\text{terminal-event})$$

is the predicate “this session has some terminal event.”

15.6 Universal quantification as a right adjoint

In a topos, inverse image also has a right adjoint:

$$f^* \dashv \forall_f.$$

In **Set**,

$$\forall_f(P) = \{b \in B : \forall a, f(a) = b \Rightarrow a \in P\}.$$

For $f : \text{Event} \rightarrow \text{Session}$, $\forall_f(P)$ is the set of sessions all of whose events satisfy P .

The adjunction law is

$$f^{-1}(Q) \subseteq P \iff Q \subseteq \forall_f(P).$$

Quantifiers therefore arise from changing context along a projection. If $\pi : X \times Y \rightarrow X$, then

$$\exists_\pi$$

and

$$\forall_\pi$$

interpret quantification over Y .

15.7 Parameter sufficiency revisited

Let $r : X \rightarrow P$ forget hidden state. Pullback

$$r^* : \text{Sub}(P) \rightarrow \text{Sub}(X)$$

turns parameter predicates into full-state predicates.

A full invariant $I \hookrightarrow X$ is exactly expressible from parameters when it lies in the image of r^* :

$$I = r^*(J)$$

for some $J \hookrightarrow P$.

The approximations

$$\exists_r(I)$$

and

$$\forall_r(I)$$

have distinct meanings:

- $\exists_r(I)$: parameters admit at least one completion satisfying I ;
- $\forall_r(I)$: every completion satisfying those parameters obeys I .

For transaction safety, $\forall_r(I)$ is the relevant guarantee. Existential satisfiability is insufficient.

15.8 Pullback functors between slice categories

For $f : A \rightarrow B$, pulling back bundles over B yields bundles over A :

$$f^* : \mathcal{C}/B \rightarrow \mathcal{C}/A.$$

In a topos this functor has both adjoints:

$$\Sigma_f \dashv f^* \dashv \Pi_f.$$

This is the fundamental theorem emphasized by Goldblatt.

- Σ_f aggregates or composes dependent data along f ;
- f^* reindexes it;
- Π_f forms dependent products or families of local sections.

Dependent types are close to this picture: a bundle $p : E \rightarrow B$ is a family of fibers E_b , and reindexing substitutes base values.

15.9 Unit and counit as round-trip maps

Every adjunction has a unit

$$\eta : 1_{\mathcal{C}} \Rightarrow GF$$

and counit

$$\varepsilon : FG \Rightarrow 1_{\mathcal{D}}$$

satisfying triangle identities.

These are not generally isomorphisms. Adjunction is weaker than equivalence.

In software, a parse/pretty-print pair often forms an approximation pattern rather than an isomorphism:

- parse after print may recover the semantic object exactly;
- print after parse may canonicalize the source.

This resembles a reflection, a special adjunction where one side embeds a subcategory of canonical objects.

15.10 Exercises

15.1 [proof]. Derive the free-monoid adjunction between sets and monoids.

15.2 [calculation]. For a finite map $f : A \rightarrow B$ and subset $P \subseteq A$, compute $\exists_f(P)$ and $\forall_f(P)$.

15.3 [design]. Let `Event` $\rightarrow$ `Session` be the scope map. Define event predicates whose existential and universal pushforwards answer useful operational questions.

15.4 [proof]. Verify both adjunction laws for inverse image, existential image, and universal image in `Set`.

15.5 [design]. For a REST parameter map, interpret $\exists_r(I)$, $\forall_r(I)$, and exact descent $I = r^*(J)$.

15.6 [research]. Identify a canonicalization pipeline in your software. Model it as a reflection if possible, stating the unit, fixed objects, and universal property.

Chapter 16

Geometric Morphisms and Architectural Translation

Source alignment: Goldblatt, Chapter 16: preservation and reflection, geometric morphisms, internal and geometric logic, and theories as sites.

16.1 Preservation and reflection

A functor preserves a construction when applying the functor to a construction in the source yields the corresponding construction in the target. It reflects a property when the property of the image implies the property in the source.

Examples:

- a left-exact functor preserves finite limits;
- a faithful functor can reflect equality of arrows;
- an equivalence preserves and reflects all categorical properties expressible up to isomorphism.

For software translations, preservation questions are concrete:

- Does encoding preserve products of independent fields?
- Does replay preserve event order?
- Does a redaction functor preserve pullback-defined authorization predicates?
- Does a client projection preserve the equalizer of live and replay semantics?

A translation can be useful while failing to preserve important constructions. The failed preservation identifies information loss.

16.2 Geometric morphisms

Definition. A geometric morphism from a topos $\mathcal{E}$ to a topos $\mathcal{F}$ consists of an adjunction

$$f^* : \mathcal{F} \rightleftarrows \mathcal{E} : f_*$$

where f^* is left adjoint to f_* and preserves finite limits.

The functor f^* is called inverse image and f_* direct image. The direction of the named geometric morphism is opposite to the inverse-image functor.

Finite-limit preservation means inverse image preserves finite contexts and equations. This is why geometric logic is stable under geometric morphisms.

16.3 Software analogy: changing worlds of observation

Suppose $\mathcal{F}$ models backend semantic contexts and $\mathcal{E}$ models browser-visible contexts. A translation may pull backend predicates and objects into the browser world while a right adjoint aggregates browser observations back into backend descriptions.

This is only a geometric morphism if the selected categories are topoi, the functors form an adjunction, and inverse image preserves finite limits. Ordinary serialization adapters should not be called geometric morphisms without this structure.

The analogy nevertheless suggests good questions:

- Which finite-limit invariants survive transport?
- Does the browser representation preserve matching of name and descriptor?
- Are conjunctions and equalities preserved?
- Which existential facts can be transported?
- Which information is visible only after direct image aggregation?

16.4 Geometric logic

Geometric formulas are built using:

- finite conjunctions;
- arbitrary disjunctions, subject to size conditions;
- existential quantification;
- equality;
- truth and falsehood.

They avoid unrestricted implication, negation, and universal quantification. Geometric formulas are preserved by inverse-image functors of geometric morphisms.

For distributed software, geometric logic matches a class of positive, locally witnessable claims:

- some event with a property exists;
- one of a family of explicit outcomes occurred;
- several equalities and predicates hold together.

Claims involving absence, global universal guarantees, or negation require more care under change of context.

16.5 Theories as sites

A logical theory can generate a category of syntactic contexts and arrows, then a topology describing which families of formulas cover another. Sheaves on this site form a classifying topos for the theory.

The deep idea is:

A theory has a geometric space of models, even when no ordinary point-set space is given.

For software, one can imagine a specification theory whose:

- objects are finite interface contexts;
- arrows are substitutions or refinements;
- covers are collectively sufficient case splits or observations;
- sheaves are implementations that assign compatible data and glue it.

This is a research direction, not a claim that SessionStream currently has a classifying topos.

16.6 Refinement as a functor with laws

The SessionStream verification work distinguishes pure transition kernels from Go runtime shells and asks for trace inclusion or abstraction mappings. A categorical formulation begins with a functor-like interpretation

$$\alpha : \text{ConcreteTraces} \rightarrow \text{AbstractTraces}$$

that preserves identity and concatenation.

Safety refinement then asks that every concrete trace map to an allowed abstract trace. Stronger claims ask preservation of limits, observations, or accepted-work custody.

Naturality appears when abstraction commutes with subsystem projection:

$$\alpha(\text{concrete trace restricted to component}) = \text{abstract trace restricted to component}.$$

This connects formal verification to the presheaf language: refinement should be a morphism of observation presheaves, not merely a final-state map.

16.7 Exercises

16.1 [design]. Choose a SessionStream translation such as protobuf encoding, WebSocket framing, or client projection. List finite-limit constructions it preserves and fails to preserve.

16.2 [proof]. Show that a functor preserving terminal objects and pullbacks preserves all finite limits.

16.3 [recognition]. Classify five system properties as geometric or non-geometric in form.

16.4 [research]. Sketch a site for a small event-driven protocol theory. Define contexts, arrows, and covers.

16.5 [design]. Define an abstraction map from concrete WebSocket traces to an abstract hydration state machine. State identity, composition, and restriction-naturality laws.

Chapter 17

From Covers to Multidimensional Shapes

Supplemental material: Goldblatt develops presheaves, sheaves, sites, and local truth but does not introduce cohomology. This chapter begins the additional route requested for connecting those ideas to topological shapes.

17.1 The nerve of a cover

Let $\{U_i\}_{i \in I}$ be a cover. Its **nerve** is a simplicial complex built from overlap data.

- A vertex i represents one context U_i .
- An edge (i, j) exists when $U_i \cap U_j$ is relevant or nonempty.
- A triangle (i, j, k) exists when the three contexts have a joint overlap.
- A tetrahedron exists when four contexts have a joint overlap.
- Higher simplices continue the pattern.

The dimension records the order of joint compatibility, not physical dimension.

For an architecture cover:

- vertices can be EventStore, Timeline, Snapshot, Live UI, and Client;
- edges record shared facts or comparison contracts;
- triangles record contexts where three components can be jointly witnessed;
- missing faces record absent higher-order witnesses.

17.2 Why triangles matter

Three pairwise comparisons do not necessarily imply one three-way compatible state.

Suppose:

- EventStore and Timeline agree under schema version v_1 ;

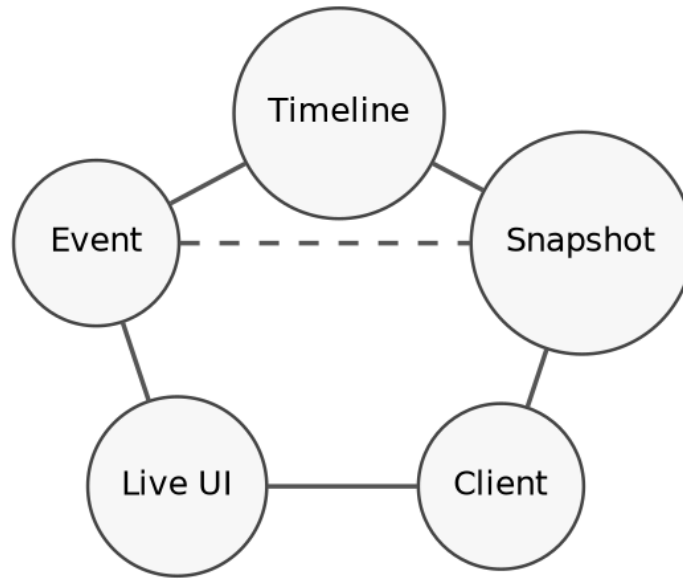


Figure 17.1: A possible nerve of SessionStream observation contexts. Edges record overlaps; a missing face can leave a cycle.

- Timeline and Snapshot agree under version v_2 ;
- Snapshot and EventStore agree after a migration.

Every edge can pass its own check while no single versioned triple realizes all three. An actual three-way observation context supplies the triangle that records joint compatibility.

A triangle is therefore not decorative. It says the boundary loop is filled by a higher-order witness.

17.3 Cycles and holes

A graph cycle is a closed path of pairwise overlaps. If the cycle is not the boundary of included higher-dimensional faces, it can support circulation data that cannot be reduced to local vertex assignments.

The intuitive word *hole* means that a compatible-looking boundary lacks an interior witness in the chosen complex.

This statement is model-relative. Adding a transaction or joint observer can add a simplex and fill a hole. Forgetting joint evidence can remove a simplex and create one.

17.4 Architecture as a simplicial complex

A disciplined construction is:

1. Choose a target invariant.

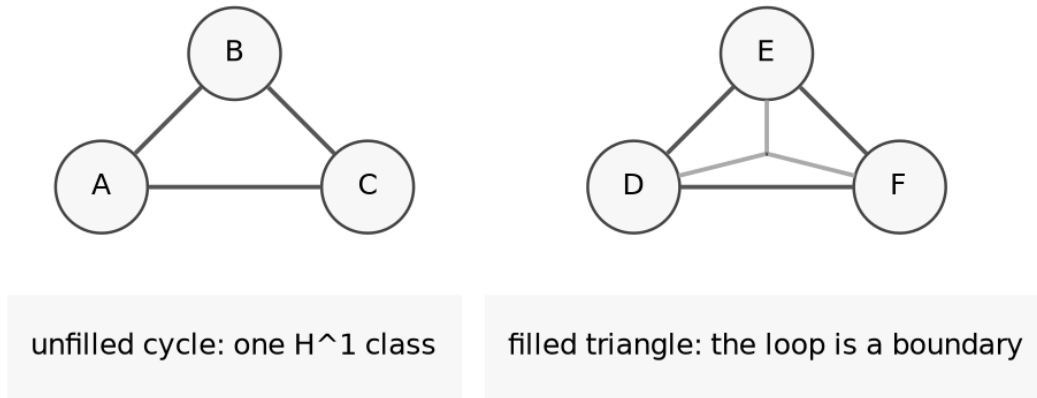


Figure 17.2: An unfilled cycle can support a first cohomology class; filling it supplies a higher-order compatibility condition.

2. List the minimal contexts that observe parts of it.
3. Add an edge when two contexts share data with a comparison map.
4. Add a higher simplex only when there is a real joint observation or compatibility witness.
5. Attach local values to every simplex through a presheaf or cellular sheaf.

Do not add a triangle merely because all three edges exist. That would assume the theorem you want to test.

17.5 Example: snapshot consistency complex

Choose vertices:

E = event-store cut,
 P = projection checkpoint,
 S = snapshot cut,
 L = maximum entity last-event coordinate.

Edges represent comparison contracts:

- $E - P$: projector progress relative to durable events;
- $P - S$: materialization progress relative to snapshot cut;
- $S - L$: entities represented by the snapshot;
- $L - E$: entity provenance relative to event history.

A transaction that atomically reads or commits E, P, S, L can justify a filled higher-dimensional cell. Without it, the graph may be only a cycle of pairwise observations taken at different instants.

The shape itself warns you that pairwise tests may miss a global race.

17.6 The nerve is not the data

The nerve records where overlaps exist. A sheaf or coefficient system records what values live over vertices, edges, and faces and how they restrict.

Two systems can have the same nerve but different local data and different cohomology. Conversely, the same values attached to a different complex can behave differently because cycles have been filled or opened.

Topology studies the shape of compatibility. Sheaf theory studies values varying over that shape.

17.7 Refining the cover

You may split one context into smaller contexts or add more observers. This refines the cover.

A good invariant should not depend arbitrarily on the granularity of observation. Čech cohomology often studies behavior under refinements and, in suitable settings, stabilizes to sheaf cohomology.

For software, compare:

- one coarse “WebSocket” context;
- separate reader, writer, heartbeat, request-worker, and observer-dispatch contexts.

The refined cover can reveal cycles and missing joint witnesses hidden by the coarse box.

17.8 Exercises

17.1 [design]. Build a nerve for the core SessionStream pipeline. Add an edge only when you can name the shared data and restriction maps.

17.2 [design]. Identify three contexts that are pairwise comparable but lack one joint atomic witness. Draw the unfilled triangle.

17.3 [research]. Explain how adding a database transaction changes the complex rather than merely changing a value on an existing vertex.

17.4 [calculation]. For a cover of four contexts, list all possible vertices, edges, triangles, and tetrahedra. Mark which exist in your architecture.

17.5 [lab]. Generate a simplicial complex from a trace schema: add a simplex whenever one trace record jointly contains the corresponding context IDs.

Chapter 18

Cochains, Coboundaries, and Cohomology

18.1 Why algebra enters

A set-valued sheaf can express gluing, but subtraction of discrepancies is unavailable. Cohomology requires coefficients with algebraic structure, usually abelian groups, modules, or vector spaces.

We therefore choose a sheaf $\mathcal{F}$ of abelian groups. Examples of additive diagnostics include:

- ordinal offsets;
- signed count differences;
- parity bits;
- conservation-law residuals;
- log-probability or cost differences in linearized models;
- vector-clock deltas after embedding into a module.

Business states themselves are rarely abelian groups. Cohomology usually applies to a diagnostic abstraction of them.

18.2 Čech cochains

For a cover $\mathcal{U} = \{U_i\}$, define

$$C^0(\mathcal{U}, \mathcal{F}) = \prod_i \mathcal{F}(U_i),$$

$$C^1(\mathcal{U}, \mathcal{F}) = \prod_{i < j} \mathcal{F}(U_i \cap U_j),$$

$$C^2(\mathcal{U}, \mathcal{F}) = \prod_{i < j < k} \mathcal{F}(U_i \cap U_j \cap U_k),$$

and so forth.

A 0-cochain assigns a local value to each context. A 1-cochain assigns a value to each overlap. A 2-cochain assigns a value to each triple overlap.

18.3 The first coboundary

For a 0-cochain $x = (x_i)$, define the edge discrepancy

$$(\delta^0 x)_{ij} = x_j|_{U_i \cap U_j} - x_i|_{U_i \cap U_j}.$$

If $\delta^0 x = 0$, the local values form a matching family.

Thus

$$H^0 = \ker \delta^0$$

is the group of compatible local sections, and for a sheaf it corresponds to global sections over the covered region.

18.4 The second coboundary

For a 1-cochain $r = (r_{ij})$, on an oriented triple $i < j < k$,

$$(\delta^1 r)_{ijk} = r_{jk} - r_{ik} + r_{ij},$$

with all terms restricted to the triple overlap.

This is the circulation around the triangle boundary. A 1-cochain is a cocycle when

$$\delta^1 r = 0.$$

Every discrepancy produced from vertex values is automatically a cocycle:

$$\delta^1 \delta^0 = 0.$$

The cancellation is algebraic: every vertex term appears twice with opposite signs.

18.5 First cohomology

The first cohomology group is

$$H^1 = \frac{\ker \delta^1}{\operatorname{im} \delta^0}.$$

- $\ker \delta^1$ contains edge assignments satisfying all triangle consistency conditions.
- $\operatorname{im} \delta^0$ contains edge assignments explained by choosing local vertex coordinates.
- A nonzero class is a locally consistent circulation that cannot be removed by changing local coordinates.

This is the precise version of “go around a loop and return shifted.”

18.6 A cycle graph

Take five contexts arranged in a cycle, with coefficients in $\mathbb{R}$. Orient edges

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0.$$

A 0-cochain is a vector

$$x = (x_0, x_1, x_2, x_3, x_4).$$

Its coboundary is

$$\delta^0 x = \begin{bmatrix} -1 & 1 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \\ 1 & 0 & 0 & 0 & -1 \end{bmatrix} x.$$

There are no filled triangles, so $C^2 = 0$ and every 1-cochain is a cocycle. The matrix has rank 4. Therefore

$$\dim H^1 = 5 - 4 = 1.$$

The invariant coordinate is total circulation:

$$r_{01} + r_{12} + r_{23} + r_{34} + r_{40}.$$

Every gradient $\delta^0 x$ has zero total circulation. A nonzero sum cannot be explained by local coordinates.

18.7 A filled triangle

For vertices 0, 1, 2 with the triangle included, a 1-cochain r must satisfy

$$r_{12} - r_{02} + r_{01} = 0$$

to be a cocycle.

The face enforces zero circulation. The boundary loop is filled, and for constant coefficients on one filled triangle,

$$H^1 = 0.$$

This is the algebraic meaning of adding a genuine joint compatibility witness.

18.8 Higher cohomology

In general,

$$H^n = \frac{\ker \delta^n}{\text{im } \delta^{n-1}}.$$

- H^0 : global compatible sections or degrees of freedom.
- H^1 : loop-like twisting and first-order gluing obstructions.
- H^2 : higher coherence obstructions over shells of triangles or 2-dimensional boundaries.

The interpretation depends on the sheaf. It is incorrect to assign a universal software meaning such as “ H^1 equals bugs.” Cohomology reports structure in the chosen coefficient system.

18.9 Sheaf cohomology versus ordinary topology

Ordinary simplicial cohomology uses constant coefficients attached uniformly to the space. Sheaf cohomology lets coefficient groups vary by context with nontrivial restriction maps.

For SessionStream, the EventStore-Timeline overlap may expose different diagnostic coordinates than the Snapshot-Client overlap. A cellular sheaf can attach different vector spaces to vertices and edges and linear restriction maps between them.

This is more expressive than a constant-value graph, and often the correct model for heterogeneous components.

18.10 Obstruction logic and its limits

In some sheaf-theoretic formulations, a nonzero cohomology class certifies that a local family cannot be extended globally. But several cautions are essential.

1. The obstruction may be sufficient but not necessary.
2. Linearization can forget nonlinear conflicts.
3. Vanishing cohomology does not guarantee a global section for arbitrary set-valued constraints.
4. A bad choice of cover or coefficients can hide the relevant problem.
5. A nonzero class may reflect an intended feature, such as a legitimate phase or gauge freedom, not a defect.

The reliable workflow is:

set-valued gluing problem $\rightarrow$ diagnostic linearization $\rightarrow$ cohomology as one source of evidence.

18.11 Exercises

18.1 [calculation]. Compute H^0 and H^1 over $\mathbb{R}$ for a path of four vertices.

18.2 [calculation]. Compute them for a square cycle with no face.

18.3 [calculation]. Add a diagonal and two filled triangles to the square. Recompute H^1 .

18.4 [proof]. Verify directly that $\delta^1\delta^0 = 0$ on a triangle.

18.5 [design]. Choose an additive diagnostic for SessionStream and define coefficient groups on contexts and overlaps.

18.6 [research]. Give a nonlinear global-section failure that is invisible after reducing all values modulo 2.

Chapter 19

Cohomological Case Studies for SessionStream

19.1 Case study: ordinal convention holonomy

Suppose five adapters use local coordinates for the same logical cut:

- x_E : EventStore cursor;
- x_P : projection checkpoint;
- x_S : snapshot cut;
- x_W : WebSocket suffix boundary;
- x_C : client last-seen coordinate.

On each overlap, an adapter declares an offset

$$r_{ij} = x_j - x_i.$$

If all offsets arise from actual local coordinates, the sum around every architecture loop must be zero.

Imagine these conventions:

EventStore:	last included event
Projection:	last included event
Snapshot:	last included event
WebSocket:	next event expected
Client:	last included event

The Snapshot-to-WebSocket conversion introduces +1, but a later adapter treats WebSocket’s value as already “last included” and introduces no compensating −1. Around the loop, total circulation is +1.

Every pairwise adapter can pass local examples. Globally, the coordinate returns shifted. This is

a nontrivial H^1 class in the cycle model.

The engineering fix can take several forms:

- type `LastIncludedOrdinal` and `NextExpectedOrdinal` separately;
- make every edge conversion explicit;
- add a joint protocol witness that fills the loop and enforces zero circulation;
- remove an unjustified comparison edge.

19.2 Case study: atomic projection progress

Facts include:

E = event appended through n ,

T = timeline materialized through n ,

P = projection cursor equals n .

A crash can produce

```
event cursor      = n
timeline state    = n
projection cursor  = n-1
```

or another partial combination depending on commit order.

This is first a global-section problem: does one committed-state object restrict to these local observations under the declared transaction semantics?

A linear cohomology model can track coordinate differences, but it does not by itself capture atomicity, crash timing, or the distinction between durable and attempted effects. The correct base model should include transaction states or a recovery state machine. Cohomology may then diagnose loops of progress claims, but it does not replace the state-machine proof.

19.3 Case study: consistent-cut snapshots

The safety condition is

$$\forall x \in \text{Entities}(S), \quad \text{LastEventOrdinal}(x) \leq \text{SnapshotOrdinal}(S).$$

This is an inequality constraint, not naturally an abelian equality. A set-valued or order-valued sheaf is the primary model.

One can derive additive residuals

$$d_x = \text{LastEventOrdinal}(x) - \text{SnapshotOrdinal}(S).$$

Safety requires $d_x \leq 0$. Cohomology of residual differences might help locate inconsistent cycles among readers, but positivity is order structure, not captured by group cohomology alone.

This case teaches when not to force cohomology onto a problem.

19.4 Case study: stable retry identity

Consider contexts:

- producer logical event;
- bus message;
- consumed event;
- persisted event row;
- projected entity update.

Each overlap carries some subset of:

`EventId`
`MessageId`
`SessionId`
`StreamId`
`Ordinal`
`PayloadDigest`

A global section is one coherent identity assignment across the delivery path.

If the current model has no `EventId`, two consumed deliveries with different ordinals can both extend the same producer event. This may be non-unique gluing rather than a cohomological obstruction: the local coordinates simply do not distinguish logical occurrence from delivery occurrence.

Add the missing coordinate first. Then study whether overlap transformations preserve it.

19.5 Case study: schema registry as a pullback, not a hole

The condition

$$\text{registryDescriptor}(\textit{name}) = \text{payloadDescriptor}(\textit{payload})$$

is a pullback constraint. There is no need for cohomology when one local equality check and its universal object solve the problem.

Cohomology becomes relevant when schema information is distributed around a cover with loops of migrations, aliases, or version translations. A cycle of type-name transformations can carry a

nontrivial automorphism even when each edge is locally valid.

19.6 Case study: observer traces

Suppose separate observers record:

- connection lifecycle;
- request decoding;
- hydration transitions;
- fanout;
- write completion;
- observer-queue admission and drops.

A presheaf assigns compatible partial traces to subsystem contexts. A global section is one execution trace whose projections equal every observer trace.

Potential failures include:

- missing events: no global trace projects to all logs;
- ambiguous ordering: several global traces fit the same partial orders;
- clock offsets: additive edge data have nonzero circulation;
- observer drops: explicit gaps mean the cover is not complete unless drop evidence is included.

Vector clocks, interval orders, and happens-before relations are often better primary coefficients than scalar timestamps. Linear embeddings can then support cohomological diagnostics.

19.7 A worked four-cycle

Take contexts E, P, S, C and observed relative offsets:

$$r_{EP} = 0, \quad r_{PS} = 0, \quad r_{SC} = 1, \quad r_{CE} = 0.$$

The total circulation is

$$0 + 0 + 1 + 0 = 1.$$

No vertex coordinates x_E, x_P, x_S, x_C satisfy all equations

$$x_P - x_E = 0,$$

$$x_S - x_P = 0,$$

$$x_C - x_S = 1,$$

$$x_E - x_C = 0.$$

Adding the equations gives $0 = 1$. The 1-cochain is non-exact.

A diagnostic should report both:

- the nonzero class or circulation value;
- one supporting cycle, here $E \rightarrow P \rightarrow S \rightarrow C \rightarrow E$.

The cycle is more actionable than a bare matrix rank.

19.8 What a useful tool should say

Bad diagnostic:

`H1 dimension = 1`

Useful diagnostic:

`No coherent ordinal coordinate exists.`

`Cycle: EventCursor -> ProjectionCursor -> SnapshotCut -> ClientCursor -> EventCursor`

`Accumulated offset: +1`

`Likely convention mismatch: SnapshotCut is "last included" while ClientCursor is treated as bo`

Mathematics should compress evidence without hiding the architecture.

19.9 Exercises

19.1 [calculation]. Solve the four-cycle equations above directly and derive the contradiction.

19.2 [design]. Build an ordinal-convention graph for SessionStream. Label every edge with a conversion function and linearized offset.

19.3 [research]. Determine which projection-progress failures are representable as additive cocycles and which require a transition-system model.

19.4 [design]. Define a global-section identity model for retries, including producer, bus, store, and projector contexts.

19.5 [lab]. From a recorded trace, construct edge timestamp offsets and search for cycles with nonzero circulation. Account for clock uncertainty intervals.

Chapter 20

Building a Local-to-Global Checker

20.1 Two layers, not one

A practical checker should have two separate engines.

20.1.1 Layer A: set-valued consistency

This layer handles arbitrary values and constraints.

- contexts and sections;
- restriction functions;
- covers and overlaps;
- matching-family checks;
- global-section search through SAT, SMT, CSP, database queries, or explicit enumeration.

20.1.2 Layer B: additive diagnostics

This layer handles abelian-group or vector-space abstractions.

- incidence and restriction matrices;
- coboundary operators;
- cocycle checks;
- exactness solving;
- basis vectors and supporting cycles.

Layer B is an explanation and obstruction tool for selected diagnostics. It must not silently replace Layer A.

20.2 A declarative model

A compact input format might be:

```
contexts:  
- id: event
```

```

    fields: [session, event_cursor]
  - id: projection
    fields: [session, projection_cursor]
  - id: snapshot
    fields: [session, snapshot_cut]
  - id: client
    fields: [session, client_cursor]

overlaps:
  - [event, projection]
  - [projection, snapshot]
  - [snapshot, client]
  - [client, event]

faces: []

coefficients:
  type: integers
  edge_value: ordinal_offset

```

A richer version describes restriction code, schemas, allowed covers, and trace provenance.

20.3 Verifying presheaf laws

Before gluing, test restriction itself.

For every section $s \in F(U)$:

$$\text{res}_{U,U}(s) = s.$$

For every chain $W \rightarrow V \rightarrow U$:

$$\text{res}_{U,W}(s) = \text{res}_{V,W}(\text{res}_{U,V}(s)).$$

Property-based tests can generate contexts and sections. A failure means the object is not even a presheaf; sheaf checks are premature.

20.4 Checking matching families

For every cover $\{U_i \rightarrow U\}$ and local sections s_i , compare restrictions on overlaps:

$$\text{res}_{U_i, U_i \times_U U_j}(s_i) = \text{res}_{U_j, U_i \times_U U_j}(s_j).$$

Report discrepancies with:

- context IDs;
- overlap fields;
- source trace or row identifiers;
- expected versus observed values;
- schema versions and cut coordinates.

20.5 Searching for global sections

For finite domains, encode all local variables and constraints in an SMT solver. A satisfying assignment is a global section. No assignment means a genuine incompatibility relative to the model.

To test uniqueness, ask for a second distinct satisfying assignment after blocking the first.

Results are:

UNSAT	no global section
SAT-UNIQUE	exactly one global section
SAT-MULTIPLE	underdetermined
UNKNOWN	resource or theory limitation

For API sufficiency, SAT-MULTIPLE can still be invariant-sufficient if the invariant has the same value in every model. Ask the solver for two completions with opposite invariant values.

20.6 Building coboundary matrices

For a simplicial complex with constant scalar coefficients:

- enumerate vertices and oriented edges;
- construct D_0 , one row per edge, with -1 at the tail and $+1$ at the head;
- enumerate oriented triangles;
- construct D_1 , one row per triangle, with signed edge incidences;
- verify $D_1 D_0 = 0$.

Then:

- a 0-cochain x has discrepancy $D_0 x$;
- a 1-cochain r is a cocycle when $D_1 r = 0$;
- it is exact when $D_0 x = r$ has a solution.

Over a field,

$$\dim H^1 = \dim \ker D_1 - \text{rank } D_0.$$

Over integers, Smith normal form also detects torsion.

20.7 A Go-oriented sketch

```

type Complex struct {
    Vertices []string
    Edges    []OrientedEdge
    Triangles []OrientedTriangle
}

type Diagnostic struct {
    EdgeValues map[OrientedEdge]int64
}

type CohomologyResult struct {
    IsCocycle      bool
    IsExact        bool
    TriangleErrors []TriangleResidual
    CycleWitnesses []CycleResidual
}

```

The checker pipeline is:

```

validate complex
-> build D0 and D1
-> assert D1*D0 == 0
-> check D1*r == 0
-> solve D0*x == r
-> if unsolved, extract a cycle witness

```

For floating timestamps, use rational values, exact integer ticks, or tolerance-aware interval constraints. Naive floating-point rank can create false classes.

20.8 Cellular sheaves for heterogeneous data

A more realistic checker attaches a vector space $F(v)$ to each component and $F(e)$ to each overlap, with linear restriction maps

$$\rho_{v,e} : F(v) \rightarrow F(e).$$

The degree-zero coboundary on edge $e = (u, v)$ is

$$(\delta x)_e = \rho_{v,e}(x_v) - \rho_{u,e}(x_u).$$

This allows `EventStore` to expose `(eventCursor, streamId)` while `Snapshot` exposes `(snapshotCut, maxEntityOrdinal)`, and the overlap compares only a shared linear projection.

Constructing these maps is the main modeling work. Matrix computation is the easy part.

20.9 Integration with traces

Each diagnostic should retain provenance:

```
value
context
source record ID
wall-clock interval
logical cut
schema version
confidence or uncertainty
```

When a class is detected, map the algebraic support back to trace records. A cycle witness should be navigable to code paths and timestamps.

This aligns with SessionStream’s observer and flight-recorder research: traces are evidence, and algebra is a projection of that evidence.

20.10 A staged research program

1. **Manual model:** draw one context cover and define restrictions.
2. **Presheaf tests:** property-test identity and composition of restriction.
3. **Matching checker:** validate pairwise overlaps in traces.
4. **Global-section solver:** encode one finite invariant in SMT.
5. **Constant-coefficient cohomology:** detect ordinal-offset loops.
6. **Cellular sheaf:** use heterogeneous component coordinates.
7. **Runtime integration:** emit trace records with model version and context IDs.
8. **Refinement evidence:** connect concrete records to an abstract transition model.

Do not begin with a generic category-theory framework. Begin with one invariant and one trace family.

20.11 Capstone project

Build `sessionstream-gluecheck` with two commands.

```
sessionstream-gluecheck sections trace.json model.yaml
sessionstream-gluecheck cohomology trace.json complex.yaml
```

The first reports existence and uniqueness of global sections for snapshot-plus-suffix custody. The second reports ordinal-coordinate cocycles and supporting cycles.

A successful capstone report should contain:

- the exact model version;

- contexts and cover used;
- restriction-law test results;
- local mismatch table;
- global-section status;
- cohomology status;
- minimal or near-minimal witness trace;
- caveats about hidden coordinates and linearization.

20.12 Exercises

20.1 [design]. Write a YAML model for snapshot-plus-suffix completeness.

20.2 [lab]. Implement restriction-law property tests for one presheaf.

20.3 [lab]. Build D_0 and D_1 for a square with one diagonal and two faces. Verify $D_1 D_0 = 0$.

20.4 [lab]. Solve exactness of an observed edge-offset vector. Return a vertex-coordinate assignment or a cycle witness.

20.5 [research]. Compare SMT global-section search with cohomology for the same trace. Document one conflict detected by only one method.

20.6 [capstone]. Implement the two-command checker above for a reduced SessionStream trace format.

Appendix A

Proof and Modeling Patterns

A.1 Proving a category

To show proposed objects and arrows form a category:

1. Define domain and codomain.
2. Define which pairs compose.
3. Prove the composite is still an allowed arrow.
4. Define identities and prove they are allowed.
5. Prove left and right identity laws.
6. Prove associativity under the chosen arrow equality.

For side-effecting software, step 6 often exposes the real modeling problem. If observable traces differ by parenthesization because resource acquisition or failure handling differs, the arrows or equality are wrong for an ordinary category.

A.2 Proving a universal property

To prove (L, λ_i) is a limit:

1. **Cone:** verify all required triangles commute.
2. **Existence:** take an arbitrary cone (X, x_i) and construct $u : X \rightarrow L$.
3. **Factorization:** verify $\lambda_i \circ u = x_i$ for every component.
4. **Uniqueness:** assume $v : X \rightarrow L$ has the same factorization equations and prove $v = u$.

Do not stop after constructing an object satisfying compatibility equations. Universality and uniqueness are the theorem.

A.3 Uniqueness up to unique isomorphism

Suppose (L, λ_i) and (M, μ_i) are both limits.

- Universality of L gives a unique $f : M \rightarrow L$.

- Universality of M gives a unique $g : L \rightarrow M$.
- Both $f \circ g$ and 1_L are arrows $L \rightarrow L$ with the required cone factorization, so uniqueness gives $f \circ g = 1_L$.
- Similarly $g \circ f = 1_M$.

Thus $L \cong M$, with the isomorphism uniquely compatible with the cones.

Use the same template dually for colimits.

A.4 Proving functoriality

For a proposed functor F :

1. verify source and target types of $F(f)$;
2. verify $F(1_A) = 1_{F(A)}$;
3. verify $F(g \circ f) = F(g) \circ F(f)$.

For restriction systems, the third law is path independence. Direct restriction and staged restriction must agree.

A.5 Proving naturality

For $\eta : F \Rightarrow G$, take an arbitrary arrow $f : A \rightarrow B$ and prove

$$G(f) \circ \eta_B = \eta_A \circ F(f).$$

Read the two paths in software language. For migration:

old state at A -> migrate -> new state at A -> evolve
must equal

old state at A -> evolve -> old state at B -> migrate

A counterexample is usually one event or transformation that migration handles nonuniformly.

A.6 Proving presheaf laws

For each context U , define $F(U)$. For each arrow $V \rightarrow U$, define

$$\rho_{U,V} : F(U) \rightarrow F(V).$$

Then prove:

$$\rho_{U,U} = 1_{F(U)},$$

$$\rho_{U,W} = \rho_{V,W} \circ \rho_{U,V}.$$

The second equation is the restriction equivalent of migration path independence.

A.7 Proving the sheaf condition

For a cover $\{U_i \rightarrow U\}$:

1. take a matching family $s_i \in F(U_i)$;
2. construct a candidate amalgamation $s \in F(U)$;
3. prove $s|_{U_i} = s_i$;
4. assume t has the same restrictions;
5. prove $t = s$.

To disprove sheafness, provide either:

- a matching family with no amalgamation;
- two distinct amalgamations with the same local restrictions.

A.8 Computing low-dimensional cohomology

For a finite simplicial complex over a field:

1. choose orientations for edges and faces;
2. enumerate bases of C^0, C^1, C^2 ;
3. construct matrices D_0, D_1 ;
4. verify $D_1 D_0 = 0$;
5. compute $\ker D_0$ for H^0 ;
6. compute $\ker D_1$ and $\text{im } D_0$;
7. obtain

$$\dim H^1 = \dim \ker D_1 - \text{rank } D_0.$$

For one observed 1-cochain r :

1. check $D_1 r = 0$;
2. solve $D_0 x = r$;
3. if no solution exists, extract a dual or cycle witness.

A.9 Model audit questions

Before trusting a categorical or cohomological result, ask:

- Which runtime distinctions were quotiented away?

- Is arrow equality observational, semantic, or byte-level?
- Are all relevant inputs explicit coordinates?
- Does the cover genuinely provide sufficient observation?
- Are higher-order joint contexts represented as faces?
- Is the coefficient system additive for a principled reason?
- Could a nonzero class be an intended degree of freedom?
- Does vanishing merely show the linear abstraction is consistent?

Appendix B

Selected Exercise Hints and Solutions

B.1 Solution to 1.2

Concatenation of words appends the symbols of one finite sequence after another. Both $(xy)z$ and $x(yz)$ contain the symbols of x , then y , then z , with the same indices, so they are equal.

This algebraic associativity does not prove concurrent serializability. Concurrent publication can assign or apply events in an order not represented by the chosen word, can interleave hidden effects, or can expose intermediate states. A refinement theorem must connect concrete execution to one legal word.

B.2 Solution to 2.1

For histories:

- Reflexive: $x \preceq x$ because $x\epsilon = x$.
- Transitive: if $xy = z$ and $zw = t$, then $x(yw) = t$, so $x \preceq t$.
- Antisymmetric: if $x \preceq y$ and $y \preceq x$, their lengths satisfy $|x| \leq |y| \leq |x|$, hence lengths are equal. The suffixes must be empty, so $x = y$.

Thus prefix order is a partial order.

B.3 Solution to 3.6

Let 0 and $0'$ be initial. There is a unique $f : 0 \rightarrow 0'$ and unique $g : 0' \rightarrow 0$. Both $g \circ f$ and 1_0 are arrows $0 \rightarrow 0$. Initiality says there is only one, so they are equal. Similarly $f \circ g = 1_{0'}$. Thus f and g are inverse isomorphisms.

B.4 Solution to 4.2

In a poset category, a product is a greatest lower bound. For histories under prefix order, the product of x and y is their longest common prefix.

A coproduct is a least upper bound. It exists only when one history is a prefix of the other, because two divergent histories have no common extension as literal words. When it exists, it is the longer history.

This reveals branching: the prefix poset is generally not a lattice.

B.5 Solution to 4.3

Let $e : E \rightarrow A$ equalize $f, g : A \rightarrow B$. Suppose $e \circ u = e \circ v$ for $u, v : X \rightarrow E$. The common arrow $h = e \circ u = e \circ v$ equalizes f, g . By the universal property, there is exactly one arrow $X \rightarrow E$ factoring h through e . Both u and v do so, hence $u = v$. Therefore e is monic.

B.6 Solution to 4.6

For $P = \{(a, b) : f(a) = g(b)\}$, let projections be $p_A(a, b) = a$ and $p_B(a, b) = b$. The square commutes by definition.

Given $u : X \rightarrow A$ and $v : X \rightarrow B$ with $f \circ u = g \circ v$, define

$$w(x) = (u(x), v(x)).$$

The compatibility equation ensures $w(x) \in P$. Its projections are u, v . If w' has the same projections, then for every x ,

$$w'(x) = (p_A w'(x), p_B w'(x)) = (u(x), v(x)) = w(x),$$

so $w' = w$.

B.7 Solution to 5.4

Given a predicate $\chi : X \rightarrow \{0, 1\}$, define

$$A_\chi = \{x : \chi(x) = 1\}.$$

Given a subset A , define χ_A in the usual way. These operations are inverse:

$$A_{\chi_A} = A, \quad \chi_{A_\chi} = \chi.$$

B.8 Hint for 6.2

Use observations of increasing strength.

- Client-level: normalized rendered entities only.

- Storage-level: exact entity rows, tombstones, and ordinals.
- Audit-level: event append, projector decisions, errors, and delivery attempts.

A replay can match client output while differing in audit trace because a transient projection error was retried.

B.9 Solution to 7.4

Let $I : X \rightarrow \{0, 1\}$ and $r : X \rightarrow P$. The invariant is parameter-determined exactly when

$$r(x_1) = r(x_2) \Rightarrow I(x_1) = I(x_2).$$

Then define $J(p) = I(x)$ for any x with $r(x) = p$. The condition makes this well-defined. If r is not surjective, define J arbitrarily outside its image. Then $I = J \circ r$.

B.10 Solution to 8.2

Assume $h \Vdash \varphi \Rightarrow \psi$ and $h \preceq k$. To prove $k \Vdash \varphi \Rightarrow \psi$, take any $l \succeq k$. Then $l \succeq h$. If $l \Vdash \varphi$, the implication forced at h gives $l \Vdash \psi$. Hence the implication persists.

B.11 Solution to 9.1

Let E^* be the free monoid and assign each generator $e \in E$ an endomorphism $\delta_e : S \rightarrow S$. Define

$$F(\epsilon) = 1_S,$$

$$F(e_1 \cdots e_n) = \delta_{e_n} \circ \cdots \circ \delta_{e_1}.$$

Then $F(xy) = F(y) \circ F(x)$ with the chosen execution convention. Any functor extending the generator assignment must have these values because it preserves identities and composition, proving uniqueness.

B.12 Solution to 10.3

Assume $r : X \rightarrow P$ is surjective.

If $I = J \circ r$, then equal parameter values imply equal invariant values immediately.

Conversely, assume the fiber criterion. For each $p \in P$, choose any x_p with $r(x_p) = p$ and define $J(p) = I(x_p)$. If another representative is chosen, the fiber criterion gives the same result. Then $J(r(x)) = I(x)$.

For constructive mathematics, the choice of representatives requires care. One can instead define J by the unique common value on each inhabited fiber.

B.13 Solution to 12.1

Prove

$$\text{fold}(S, xy) = \text{fold}(\text{fold}(S, x), y)$$

by induction on y .

Base $y = \epsilon$: both sides are $\text{fold}(S, x)$.

Step $y = ze$:

$$\text{fold}(S, xze) = \delta(\text{fold}(S, xz), e)$$

and by induction this equals

$$\delta(\text{fold}(\text{fold}(S, x), z), e) = \text{fold}(\text{fold}(S, x), ze).$$

B.14 Solution to 14.1

A matching family is a tuple in

$$\prod_i F(U_i)$$

whose two restrictions to every pairwise overlap agree. Therefore it lies in the equalizer of the two maps

$$\prod_i F(U_i) \rightrightarrows \prod_{i,j} F(U_i \cap U_j).$$

The sheaf condition says restriction from $F(U)$ gives a bijection onto this equalizer: every matching family has exactly one amalgamation.

B.15 Solution to 15.2

Let

$$A = \{a_1, a_2, a_3, a_4\}, \quad B = \{b_1, b_2\},$$

with $f(a_1) = f(a_2) = b_1$, $f(a_3) = f(a_4) = b_2$. Take $P = \{a_1, a_3, a_4\}$.

Then

$$\exists_f(P) = \{b_1, b_2\}$$

because each fiber contains at least one member of P .

But

$$\forall_f(P) = \{b_2\}$$

because the entire fiber over b_2 lies in P , while $a_2 \notin P$ blocks b_1 .

B.16 Solution to 18.1

For a path with four vertices and three edges, D_0 has rank 3. The complex is connected, so

$$\dim H^0 = 1.$$

There are no faces, so $\ker D_1 = C^1$ has dimension 3. Therefore

$$\dim H^1 = 3 - 3 = 0.$$

Every edge assignment is a gradient because a tree has no cycle obstruction.

B.17 Solution to 18.2

A square cycle has four vertices and four edges, is connected, and has no faces. Thus

$$\text{rank } D_0 = 3, \quad \dim C^1 = 4,$$

so

$$\dim H^1 = 4 - 3 = 1.$$

The class is measured by total oriented circulation around the square.

B.18 Solution to 18.3

Adding a diagonal and both triangular faces gives five edges and two independent face equations. The complex is a filled disk, so it is connected and has no first cohomology:

$$\dim H^0 = 1, \quad \dim H^1 = 0.$$

Algebraically, $\dim \ker D_1 = 3$ and $\text{rank } D_0 = 3$.

B.19 Solution to 18.4

For vertex values x_0, x_1, x_2 , edge differences are

$$r_{01} = x_1 - x_0, \quad r_{02} = x_2 - x_0, \quad r_{12} = x_2 - x_1.$$

Then

$$r_{12} - r_{02} + r_{01} = (x_2 - x_1) - (x_2 - x_0) + (x_1 - x_0) = 0.$$

B.20 Solution to 19.1

The equations are

$$x_P = x_E, \quad x_S = x_P, \quad x_C = x_S + 1, \quad x_E = x_C.$$

The first two give $x_S = x_E$. The third gives $x_C = x_E + 1$. The fourth gives $x_E = x_E + 1$, impossible. Equivalently, summing the four edge equations gives $0 = 1$.

Appendix C

Study Plan and Working Notebook

C.1 A twenty-session route

Session	Reading	Deliverable
1	Chapters 1-2	Define two categories from your codebase.
2	Chapter 3	Classify five arrows as monic, epic, iso, or neither.
3	Chapter 4 through equalizers	Prove one product and one equalizer universal property.
4	Rest of Chapter 4	Draw the schema pullback and a hydration limit.
5	Chapter 5	Represent four invariants as subobjects.
6	Chapters 6-7	Build an invariant lattice for snapshots.
7	Chapter 8	Build a prefix Kripke model for chat inference.
8	Chapter 9	Model event interpretation as a functor.
9	Chapter 10	Perform a parameter-sufficiency fiber analysis.
10	Chapter 11	Write a many-sorted protocol signature.
11	Chapter 12	Prove a fold invariant by induction.
12	Chapter 13	Implement one restriction system and test its laws.

Session	Reading	Deliverable
13	Chapter 14 through gluing	Specify snapshot/suffix matching conditions.
14	Rest of Chapter 14	Design a site of sufficient observations.
15	Chapter 15	Compute existential and universal pushforwards.
16	Chapter 16	Audit what one adapter preserves.
17	Chapter 17	Build an architecture nerve.
18	Chapter 18	Compute H^0, H^1 for three small complexes.
19	Chapter 19	Model an ordinal-convention cycle.
20	Chapter 20	Implement a reduced glue checker.

C.2 Notebook page template

For every new concept, fill one page with:

Definition:

Diagram:

SessionStream object model:

Exact analogy, partial analogy, or non-analogy:

One law:

One counterexample:

One executable test:

One unanswered question:

The “non-analogy” line is important. It prevents mathematical vocabulary from becoming decorative architecture language.

C.3 Diagram drills

Redraw from memory:

1. category composition and identities;
2. monic cancellation;
3. product universal property;
4. equalizer universal property;
5. pullback;
6. a general limit cone;
7. subobject classifier pullback;
8. naturality square;
9. presheaf restriction;
10. sheaf matching family and amalgamation;
11. adjunction hom-set correspondence;
12. an unfilled cycle and a filled triangle.

For each diagram, narrate the types of every arrow. A diagram whose arrows cannot be typed is not understood.

C.4 Proof-writing standard

A complete proof should state:

- arbitrary objects and arrows introduced;
- why every composite is defined;
- which law or universal property is used;
- where existence is established;
- where uniqueness is established;
- what equality of arrows means.

Avoid “obvious” until you can supply the omitted equation.

Appendix D

Glossary

Adjunction. A natural correspondence between arrows $F(A) \rightarrow B$ and $A \rightarrow G(B)$.

Amalgamation. A global section whose restrictions are a given matching family.

Arrow. A morphism with specified domain and codomain.

Cartesian closed category. A category with finite products and exponentials.

Characteristic arrow. The predicate $X \rightarrow \Omega$ classifying a subobject of X .

Cocycle. A cochain in the kernel of the next coboundary.

Coboundary. The alternating restriction map $\delta^n : C^n \rightarrow C^{n+1}$.

Cochain. An assignment of coefficient data to simplices or overlaps of one degree.

Cohomology. The quotient of cocycles by coboundaries, $H^n = \ker \delta^n / \text{im } \delta^{n-1}$.

Colimit. A universal cocone from a diagram.

Cone. A compatible family of arrows from one apex object into a diagram.

Context. The base object over which a section is defined.

Contravariant functor. A functor reversing arrow direction, equivalently a functor from an opposite category.

Coproduct. A universal sum with injections.

Cover. A family declared collectively sufficient to determine data over a context.

Epic. Right-cancellable arrow.

Equalizer. Universal subobject on which two parallel arrows agree.

Exact cochain. A cochain in the image of the previous coboundary.

Exponential. Function-object B^A characterized by currying.

Fiber. The inverse image of one value under a map; possible global completions of local data.

Functor. A structure-preserving map between categories.

Generalized element. An arrow $U \rightarrow X$ from an arbitrary context.

Geometric morphism. An adjunction between topoi whose inverse-image functor preserves finite limits.

Global section. A section over the largest context under consideration.

Grothendieck topology. A system of covering sieves satisfying maximality, pullback stability, and transitivity.

Heyting algebra. A bounded distributive lattice with implication right adjoint to meet.

Image. The smallest subobject through which an arrow factors.

Initial object. An object with a unique arrow to every object.

Isomorphism. An arrow with a two-sided inverse.

Limit. A universal cone into a diagram.

Matching family. Local sections agreeing after restriction to all overlaps.

Monic. Left-cancellable arrow.

Natural transformation. A componentwise translation between functors commuting with every source arrow.

Nerve. A simplicial complex encoding nonempty or relevant finite overlaps of a cover.

Opposite category. The category obtained by reversing every arrow.

Power object. An object classifying subobjects of products, usually Ω^A in a topos.

Presheaf. A contravariant set-valued functor on a context category.

Product. A universal pairing object with projections.

Pullback. A limit of a cospan; the object of matching pairs over a common target.

Pushout. A colimit of a span; a universal merge with specified identifications.

Restriction. The presheaf map from sections over a larger context to a smaller one.

Section. A value or assignment defined over a context.

Separated presheaf. A presheaf in which a matching family has at most one amalgamation.

Sheaf. A presheaf in which every matching family has a unique amalgamation.

Sheafification. The universal passage from a presheaf to an associated sheaf.

Sieve. A precomposition-closed family of arrows into one object.

Site. A category equipped with a Grothendieck topology.

Subobject. An equivalence class of monic arrows into one object.

Subobject classifier. An object Ω and **true** arrow classifying every subobject by pullback.

Terminal object. An object receiving a unique arrow from every object.

Topos. A Cartesian closed category with a subobject classifier.

Universal property. A characterization by unique factorization among all competing candidates.

Appendix E

Source Notes and Further Reading

E.1 Primary pedagogical source

Robert Goldblatt, *Topoi: The Categorical Analysis of Logic*, second edition with later Dover preface. The supplied EPUB was used for chapter order, definitions, conceptual progression, and exercise style. This companion cites section numbers rather than reproducing passages.

Especially important correspondences are:

- §2.3: category definition;
- §§3.1-3.16: arrow properties and universal constructions;
- §4.3: elementary topos definition;
- Chapter 8: intuitionistic logic and Kripke semantics;
- Chapter 9: functors and natural transformations;
- §14.1: presheaves, compatibility, and unique pasting;
- §§14.3-14.6: sites, geometric modality, and local truth;
- Chapter 15: adjunctions and quantifiers;
- Chapter 16: geometric morphisms and geometric logic.

E.2 SessionStream source snapshot

Repository: github.com/go-go-golems/sessionstream

Commit: `d62dca9f5efa2e3094d6c62e5ead5ed0c88fd35c`

Key observations used in examples:

- `types.go`: `SessionId`, `Command`, `Event`, and `Session`;
- `projection.go`: UI and timeline projections over events and timeline views;
- `hydration.go`: snapshots, event replay, projection cursors, and error custody interfaces;
- `hub.go`: command dispatch, local publication, projection/application order, rebuild, and fanout;
- `schema.go`: named protobuf prototype registry and payload validation;

- `ordinals.go`: per-session ordinal assignment and stream-ID-derived coordinates;
- `consumer.go`: bus consumption, ordinal assignment, project/apply, and acknowledgement;
- `hydration/sqlite/store.go`: event and entity persistence, historical versions, cursors, and snapshot reads;
- `transport/ws/server.go`: hydrating/live subscription states, buffering, snapshot send, suffix flush, heartbeat, and observer dispatch;
- `examples/chatdemo/chat.go`: event generators and separate UI/timeline interpretations.

The Architecture Garden study supplied the explicit hardening laws used throughout: per-session serializability, consistent-cut snapshots, stable retry identity, atomic projection progress, deterministic replay, and snapshot-plus-suffix completeness.

E.3 Additional mathematical references

These are optional extensions rather than sources for the chapter sequence.

- Saunders Mac Lane, *Categories for the Working Mathematician*.
- Saunders Mac Lane and Ieke Moerdijk, *Sheaves in Geometry and Logic*.
- F. William Lawvere and Stephen Schanuel, *Conceptual Mathematics*.
- David Spivak, *Category Theory for the Sciences*.
- Brendan Fong and David Spivak, *An Invitation to Applied Category Theory*.
- Justin Curry, *Sheaves, Cosheaves and Applications*.
- Robert Ghrist, *Elementary Applied Topology*.
- Samson Abramsky and Adam Brandenburger, work on sheaf-theoretic contextuality.

E.4 Copyright and scope note

This book is a newly written study aid. It is not an edition, abridgment, or substitute for Goldblatt's text. The source book remains necessary for its historical argument, detailed logical development, and original exercise sequence. The cohomology chapters are supplemental and should not be attributed to Goldblatt.

Closing Perspective

The progression of ideas can now be compressed into one line:

arrows $\rightarrow$ universal constructions $\rightarrow$ predicates and logic
 $\rightarrow$ context-dependent sets $\rightarrow$ local gluing
 $\rightarrow$ shape and obstruction.

For SessionStream, the corresponding engineering progression is:

typed events $\rightarrow$ projections and cuts $\rightarrow$ invariants
 $\rightarrow$ partial observers $\rightarrow$ snapshot-plus-suffix assembly
 $\rightarrow$ diagnostics of missing global coherence.

The most productive immediate practice is not to compute cohomology. It is to define one presheaf correctly: choose contexts, write restriction maps, and test identity and composition. Then choose one cover, state the matching condition, and determine whether amalgamations exist and are unique. Once those objects are concrete, the multidimensional shapes cease to be metaphors. They become the actual combinatorics of which parts of the system can be known together.